

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	8
1.1 Описание предметной области	8
1.2 Определение автоматизируемых задач	9
2 РАЗРАБОТКА ТЕХНИЧЕСКОГО ЗАДАНИЯ.....	10
3 ТЕХНИЧЕСКОЕ ПРОЕКТИРОВАНИЕ	17
3.1 Логическая проработка функциональной и системной архитектуры информационной системы.....	17
3.2 Выбор программного средства для разработки информационной системы.....	25
3.3 Описание базы данных	25
4 РАБОЧЕЕ ПРОЕКТИРОВАНИЕ	28
4.1 Создание базы данных	28
4.2 Создание пользовательского интерфейса	30
4.3 Создание программного продукта.....	33
4.4 Тестирование программного продукта	35
5 ИНСТРУКЦИЯ ПО РАБОТЕ С АВТОМАТИЗИРОВАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМОЙ	37
5.1 Требования к техническому обеспечению.....	37
5.2 Инструкция системному администратору, программисту	37
5.3 Руководство пользователю	39
6. БЕЗОПАСНОСТЬ И НАДЕЖНОСТЬ ФУНКЦИОНИРОВАНИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	43
ЗАКЛЮЧЕНИЕ	44
СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ	45
ПРИЛОЖЕНИЯ	48

ВВЕДЕНИЕ

Стоматологические услуги являются одними из самых востребованных в сфере здравоохранения. Современная стоматологическая клиника сталкивается с необходимостью обработки большого объема данных: расписание приёмов, карточки пациентов, медицинские истории, учёт услуг и врачей, финансовая отчётность. Традиционные бумажные журналы и разрозненные электронные таблицы не обеспечивают оперативности, достоверности и удобства доступа. Клиенты ожидают возможности записаться онлайн 24/7, а сотрудники — быстрого поиска информации и автоматического формирования документов.

Актуальность работы обусловлена потребностью стоматологических клиник в автоматизации ключевых бизнес-процессов таких как запись на приём, ведение электронной истории лечения, контроль загрузки врачей, мотивация пациентов через геймификацию (система достижений). Разработанная АИС решает эти задачи, предоставляя веб-интерфейс для пациентов, врачей, менеджеров и администраторов.

Объект исследования – деятельность стоматологической клиники по организации приёма пациентов, учёту медицинских вмешательств и управлению персоналом.

Предмет исследования – методы и средства разработки автоматизированной информационной системы с ролевой моделью, динамическим расписанием и элементами геймификации.

Цель работы – разработать и внедрить АИС для стоматологической клиники, обеспечивающую онлайн-запись, ведение медицинской истории, разграничение прав доступа, систему достижений для пациентов, адаптивный веб-интерфейс.

Задачи

1. Проанализировать предметную область и существующие аналоги.
2. Сформулировать техническое задание.

3. Спроектировать архитектуру, базу данных и ролевую модель.
4. Обосновать выбор технологий (Flask, SQLAlchemy, MySQL, HTML/CSS/JS).
5. Реализовать бэкенд на Python с использованием микрофреймворка Flask.
6. Разработать фронтенд адаптивные шаблоны, карусель, динамическую загрузку слотов, пагинацию, мобильное меню.
7. Внедрить механизм достижений (ачивок) для пациентов.
8. Провести тестирование и подготовить инструкции.

Методы исследования

Системный анализ, UML-моделирование, объектно-ориентированное программирование, клиент-серверная архитектура, реляционные базы данных, тестирование методом «чёрного ящика».

Практическая значимость – созданный программный продукт готов к внедрению в реальной стоматологической клинике, позволяет сократить время записи, исключить конфликты в расписании, повысить лояльность пациентов за счёт ачивок и удобного интерфейса.

Структура работы. Дипломная работа состоит из введения, шести глав, заключения, списка использованных источников и приложений. В первой главе проводится анализ предметной области, во второй — проектирование системы, в третьей — функциональное моделирование, в четвертой — объектное проектирование с UML-диаграммами, в пятой — описание реализации, в шестой — тестирование. В приложениях приведены листинги кода и руководства пользователя.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Описание предметной области

Стоматологическая клиника как предприятие медицинских услуг включает следующие основные процессы: регистрация пациентов, планирование приёмов, проведение лечебных процедур, ведение медицинской документации, учёт услуг и материалов, расчёт заработной платы врачей, взаимодействие с пациентами (напоминания, информирование). В типичной клинике работают врачи разных специализаций (терапевты, хирурги, ортодонты, имплантологи), администраторы (менеджеры), управляющий персонал.

В настоящее время многие стоматологии используют устаревшие методы такие как бумажные журналы записи, Excel-таблицы, телефонные звонки. Это приводит к следующим проблемам:

- двойные записи на одно время из-за человеческого фактора;
- потеря или искажение истории лечения;
- отсутствие самозаписи пациентов во внерабочее время;
- невозможность быстрого поиска пациента по критериям;
- низкая вовлечённость пациентов в процесс лечения (нет системы опощений).

Разрабатываемая АИС призвана автоматизировать эти процессы, предоставить пациентам личный кабинет с историей лечения и ачивками, врачам – расписание и доступ к медицинским картам, менеджерам – инструменты управления записями и пациентами, администратору – полный контроль над системой.

1.2 Определение автоматизируемых задач

На основе анализа деятельности стоматологической клиники выделены следующие функциональные блоки, подлежащие автоматизации:

1. Регистрация и аутентификация – создание учётных записей пациентов, вход по email/телефону и паролю, восстановление пароля через токен.
2. Ролевая модель – четыре роли, это пациент, врач, менеджер, администратор. Каждая роль имеет строго определённые права.
3. Управление записями на приём – создание, просмотр, отмена, завершение приёма. Проверка свободных временных слотов для выбранного врача и даты.
4. Электронная медицинская история – добавление диагнозов и лечения с привязкой к пациенту и врачу, хронологический просмотр, печать.
5. Учёт услуг и прайс-листа – динамическое отображение категорий, цен, поиск и фильтрация.
6. Управление справочниками – врачи, кабинеты, услуги, достижения.
7. Геймификация (ачивки) – автоматический расчёт прогресса (количество визитов, вылеченных зубов, гигиенических процедур) и разблокировка наград.
8. Статистика и аналитика – отображение ключевых показателей для каждой роли (количество приёмов сегодня, общее число пациентов, загрузка врачей).
9. Поиск и фильтрация – поиск пациентов по ФИО/телефону, фильтрация записей по статусу и дате, пагинация.
10. Мобильная адаптация – hamburger-меню, выезжающий sidebar, overlay, корректное отображение на экранах до 320px.

2 РАЗРАБОТКА ТЕХНИЧЕСКОГО ЗАДАНИЯ

1. Общие сведения

1.1. Полное наименование системы: Автоматизированная информационная система стоматологической клиники «ORDENTIST» (АИС «ORDENTIST»).

1.2. Шифр темы или номер договора 0.

1.3. Наименование предприятий (объединений) Заказчика и Разработчика:

- Заказчик: ООО «Стоматологическая клиника ORDENTIST».
- Разработчик: студент группы 0907-26 Яковлев Илья Александрович.

1.4. Перечень документов, на основании которых создается система:

- ГОСТ 34.602-2020 «Информационная технология (ИТ). Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

- Аналитическая записка по результатам обследования бизнес-процессов стоматологической клиники.

- Требования Заказчика, сформулированные в техническом предложении.

1.5. Плановые сроки начала и окончания работы по созданию системы: начало – 01.03.2026, окончание – 31.05.2026.

1.6. Сведения об источниках и порядке финансирования работ: финансирование не требуется (разработка в рамках дипломного проекта).

2. Назначение и цели создания системы

2.1. Назначение системы

АИС «ORDENTIST» предназначена для автоматизации основных бизнес-процессов стоматологической клиники, включая: онлайн-запись пациентов на приём, ведение электронной медицинской истории, учёт услуг и врачей, управление расписанием, формирование статистической отчётности, а также мотивацию пациентов через систему достижений (геймификацию).

2.2. Цели создания системы

- Повышение эффективности и прозрачности работы администраторов и врачей.
- Сокращение времени на обработку записей за счёт самозаписи пациентов 24/7.
- Исключение конфликтных ситуаций (двойных записей) автоматической проверкой свободных слотов.
- Централизация данных о пациентах, истории лечения и услугах.
- Повышение лояльности пациентов через систему достижений и удобный личный кабинет.

3. Характеристики объектов автоматизации

К объектам автоматизации относятся:

- Пациенты – физические лица, обращающиеся за стоматологической помощью. Автоматизируется регистрация, запись на приём, просмотр истории лечения и достижений.
- Врачи – сотрудники клиники (стоматологи-терапевты, хирурги, ортодонты и др.). Автоматизируется расписание, фиксация результатов лечения (история болезни), управление статусами записей.
- Менеджеры (администраторы клиники) – сотрудники, отвечающие за запись, контроль расписания и работу с пациентами. Автоматизируется управление всеми записями, поиск пациентов, добавление врачей.
- Администратор системы – ответственный за техническую поддержку и управление пользователями. Автоматизируется создание/удаление учётных записей, назначение ролей.
- Услуги – справочник стоматологических услуг с категориями и ценами. Автоматизируется динамическое отображение прайс-листа на сайте.
- Записи на приём – основной документ, фиксирующий дату, время, врача, пациента, услугу и статус. Автоматизируется создание, изменение, отмена.

– Медицинская история – данные о диагнозах и лечении. Автоматизируется добавление, хранение и печать.

4. Требования к системе

4.1. Требования к системе в целом

4.1.1. Требования к структуре и функционированию

Система должна быть построена по трёхуровневой архитектуре «веб-клиент – сервер приложений – база данных». Доступ к системе – через веб-браузер. Система должна обеспечивать одновременную работу не менее 50 пользователей.

4.1.2. Требования к степени адаптации

Система должна позволять настраивать:

- роли и права доступа пользователей (пациент, врач, менеджер, администратор);
- справочник услуг и категорий;
- список достижений (ачивок) для пациентов.

4.1.3. Требования к надёжности

- Время наработки на отказ – не менее 500 часов.
- Время восстановления после сбоя – не более 30 минут (перезапуск приложения).
- Обязательное ежедневное резервное копирование базы данных средствами СУБД.

4.1.4. Требования к безопасности

- Разграничение прав доступа на основе четырёх ролей.
- Аутентификация по email/телефону и паролю. Пароли хранятся в хешированном виде (Werkzeug).
- Защита от SQL-инъекций через ORM SQLAlchemy.
- Защита от XSS средствами шаблонизатора Jinja2.

4.1.5. Требования к эргономике и технической эстетике

Интерфейс должен быть интуитивно понятным, адаптивным (поддержка мобильных устройств), выполненным в едином стиле (зелёная цветовая гамма,

шрифт Montserrat). Навигация – через боковое меню (sidebar) с гамбургер-кнопкой на мобильных.

4.1.6. Требования к эксплуатационной документации

Должна быть разработана следующая документация:

- руководство системного администратора (п. 5.1 диплома);
- руководство пользователя (п. 5.2 диплома).

4.2. Требования к функциям (задачам), выполняемым системой

Система должна обеспечивать выполнение следующих функций:

1. Управление пользователями и ролями (администратор):

- создание учётных записей с ролями «пациент», «врач», «менеджер», «администратор»;
- удаление пользователей;
- просмотр списка всех пользователей.

2. Регистрация и аутентификация:

- самостоятельная регистрация пациентов (ФИО, телефон, email, пароль);
- вход по email/телефону и паролю;
- восстановление пароля через временный токен.

3. Управление записями на приём:

- онлайн-запись (выбор врача, услуги, даты и свободного времени);
- динамическая подгрузка свободных слотов через API /api/slots;
- отмена записи пациентом;
- завершение/отмена записи врачом или менеджером;
- просмотр списка записей с фильтрацией по дате и статусу.

4. Ведение медицинской истории:

- добавление записей (диагноз, лечение, дата визита) врачом или менеджером;
- хронологический просмотр истории лечения;
- печать истории лечения.

5. Учёт услуг и прайс-листа:

- динамическое отображение услуг с ценами и категориями;
- поиск и фильтрация по категориям.

6. Геймификация (достижения):

- автоматический расчёт прогресса достижений на основе истории лечения;
- отображение достижений в личном кабинете пациента.

7. Статистика и аналитика:

- отображение ключевых показателей для каждой роли (количество записей сегодня, число пациентов, загрузка врачей).

8. Адаптивный веб-интерфейс:

- корректное отображение на экранах от 320px;
- мобильное меню (hamburger, drawer, overlay).

5. Состав и содержание работ по созданию системы

1. Предпроектное обследование (анализ требований, моделирование процессов).

2. Техническое проектирование (разработка ТЗ, проектирование БД и архитектуры, UML-диаграммы).

3. Рабочее проектирование (разработка пользовательского интерфейса и программных модулей).

4. Программирование и тестирование (написание кода на Flask, модульное и интеграционное тестирование).

5. Ввод системы в опытную эксплуатацию (развёртывание на локальном сервере, демонстрация Заказчику).

6. Ввод системы в промышленную эксплуатацию (перенос на сервер клиента, финальная приёмка).

7. Сопровождение системы (техническая поддержка в течение гарантийного периода – 3 месяца).

6. Порядок контроля и приёмки системы

6.1. Виды, состав, объём и методы испытаний

Проводится опытная эксплуатация в течение 2 (двух) недель на тестовом стенде. В испытаниях участвуют сотрудники клиники (менеджер, врач). Проверяется корректность выполнения всех функций, указанных в п. 4.2, а также соответствие нефункциональным требованиям (быстродействие, безопасность, адаптивность).

6.2. Общие требования к приёмке работ

Приёмка осуществляется комиссией, назначенной Заказчиком. Основанием для приёмки является успешное прохождение опытной эксплуатации, отсутствие критических ошибок и предоставление всей эксплуатационной документации (руководство пользователя, руководство администратора).

7. Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие

- Назначить ответственного за эксплуатацию системы со стороны клиники (менеджера или администратора).
- Проверить готовность рабочих мест пользователей (наличие компьютеров с доступом в локальную сеть/интернет, современных браузеров).
- Провести первичное наполнение системы справочной информацией (список врачей, услуги с ценами, категории, достижения) с помощью скрипта `init_db.py` или через административный интерфейс.

8. Требования к документированию

- Состав программной документации должен включать: техническое задание (данный документ), пояснительную записку (диплом), руководство системного администратора, руководство пользователя.
- Документация предоставляется в электронном виде (формат PDF и DOCX).

9. Источники разработки

- Результаты интервью с сотрудниками стоматологической клиники (администратором и врачами).

- Анализ существующих АИС для медицинских учреждений (YCLIENTS, Medesk, ClinicOn).
- ГОСТ 34.602-2020 «Техническое задание на создание автоматизированной системы».
- Документация по Flask, SQLAlchemy, MySQL.

3 ТЕХНИЧЕСКОЕ ПРОЕКТИРОВАНИЕ

3.1 Логическая проработка функциональной и системной архитектуры информационной системы

Диаграмма вариантов использования является одним из ключевых инструментов анализа требований в процессе проектирования информационной системы. Она позволяет отразить основные функции системы с точки зрения пользователей и определить, какие действия могут выполнять различные категории пользователей, называемых актерами.

Для автоматизированной информационной системы стоматологической клиники определены четыре основных актера, это пациент, врач, менеджер и администратор.

- Актер «Пациент» – любое физическое лицо, прошедшее регистрацию на сайте и имеющее доступ к личному кабинету. Пациент может записываться на приём, просматривать свою историю лечения и достижения.

- Актер «Врач» – сотрудник клиники, авторизованный в системе, который ведёт расписание, фиксирует результаты лечения и может просматривать карточки пациентов.

- Актер «Менеджер» – администратор клиники, управляющий записями, пациентами и справочниками (врачи, услуги, кабинеты).

- Актер «Администратор» – системный администратор, имеющий полные права на управление пользователями и ролями.

Варианты использования для актера «Пациент»

Актеру «Пациент» доступен следующий набор вариантов использования.

«Регистрация в системе» – позволяет пациенту создать учётную запись, указав имя, фамилию, телефон, email и пароль. После регистрации пациент автоматически авторизуется.

«Вход в систему» – обеспечивает доступ в личный кабинет по email/телефону и паролю. Также предусмотрен вариант «Восстановление пароля» через отправку временного токена на email (срок действия – 1 час).

«Просмотр прайс-листа» – предоставляет актуальный список стоматологических услуг с ценами и категориями (терапия, хирургия, ортодонтия, гигиена и т.д.). Данные загружаются динамически из базы данных, что гарантирует актуальность.

«Просмотр информации о врачах» – даёт возможность увидеть фотографии, квалификацию и стаж специалистов, что помогает пациенту сделать осознанный выбор.

«Запись на приём» – ключевой вариант использования, включающий несколько обязательных подчинённых вариантов:

- «Выбор врача» – из выпадающего списка, сформированного из базы данных.

- «Выбор услуги» – аналогично из справочника услуг.

- «Выбор даты» – календарь, начиная с текущей даты.

- «Выбор времени» – после выбора врача и даты система через AJAX-запрос к API /api/slots загружает свободные 30-минутные слоты (с 8:00 до 20:00), исключая уже занятые записи.

- «Подтверждение записи» – отправка формы на сервер, повторная проверка свободного времени, создание записи со статусом «scheduled». После успешного сохранения пациент получает flash-сообщение.

«Просмотр моих записей» – отображение таблицы с предстоящими и завершёнными визитами, возможностью отменить запись (статус «cancelled») в случае необходимости.

«Отмена записи» – позволяет пациенту самостоятельно отказаться от запланированного приёма.

«Просмотр истории лечения» – хронологический список всех посещений с указанием диагноза, назначенного лечения и врача. Предусмотрена возможность печати истории (через отдельное окно с CSS для печати).

«Просмотр достижений (ачивок)» – отображение прогресса по различным наградам («Первый визит», «Вылечить 5 зубов», «Чистюля» и др.). Прогресс обновляется автоматически на основе записей в медицинской истории.

«Редактирование профиля» – изменение личных данных (ФИО, телефон, email) и смена пароля.

«Выход из системы» – завершение сеанса работы.

Варианты использования для актера «Врач»

Актеру «Врач» доступны следующие варианты использования после успешной авторизации.

«Просмотр расписания» – список записей, назначенных на данного врача, с фильтрацией по дате и статусу (запланированы, завершены, отменены). Для каждой записи отображается пациент, услуга, время, кабинет.

«Завершение приёма» – изменение статуса записи на «completed» после оказания услуги.

«Отмена приёма» – изменение статуса на «cancelled» (например, при неявке пациента или переносе).

«Добавление записи в историю лечения» – заполнение формы с диагнозом, лечением и датой визита. Данные привязываются к пациенту и врачу.

«Просмотр карточки пациента» – детальная информация о пациенте (ФИО, телефон, дата рождения, полная история лечения и список всех записей). Доступно по ссылке из таблицы записей или поиска.

«Загрузка фото профиля» – возможность загрузить собственную фотографию (форматы PNG, JPG, JPEG, WEBP) для отображения на сайте.

«Смена пароля» – обновление учётных данных.

«Выход из системы» – завершение сеанса.

Варианты использования для актера «Менеджер»

Актер «Менеджер» имеет расширенные права по сравнению с врачом.

«Управление записями» – просмотр всех записей всех пациентов (не только закрепленных за конкретным врачом). Доступны фильтрация по

статусу, поиск по пациенту или врачу, пагинация (по 10 строк). Менеджер может отменить или завершить любую запись.

«Поиск пациентов» – по фамилии, имени или телефону. Результат отображается в виде таблицы с количеством визитов и датой последнего приёма, реализована пагинация.

«Просмотр полной карточки пациента» – аналогично врачу, но без ограничений.

«Добавление нового врача» – заполнение формы (ФИО, специализация, стаж, email, временный пароль). Создаётся запись в таблицах doctors и users с ролью «doctor».

«Добавление записи в историю лечения» – может дополнять историю любого пациента (в отличие от врача, который добавляет только для своих).

«Управление справочниками» (в перспективе развития) – добавление/редактирование услуг, кабинетов.

«Просмотр статистики» – общее количество записей сегодня, число врачей, пациентов, кабинетов (выводится в верхней части личного кабинета).

Варианты использования для актера «Администратор»

Актер «Администратор» обладает максимальными привилегиями.

«Управление пользователями» – просмотр списка всех пользователей системы (id, email, телефон, роль, дата регистрации). Возможно удаление любого пользователя, кроме самого себя.

«Добавление пользователя с любой ролью» – администратор может создать учётную запись для врача, менеджера или другого администратора, указав email, роль, пароль.

«Управление ролями» – при создании пользователя может выбрать любую из четырёх ролей.

«Просмотр всех записей и пациентов» – те же функции, что у менеджера, плюс возможность удалять записи (прямое удаление из БД – не реализовано, но можно через интерфейс менеджера изменить статус).

«Резервное копирование» – администратор может запустить дамп базы данных через скрипт (внешнее средство, но в документации описано).

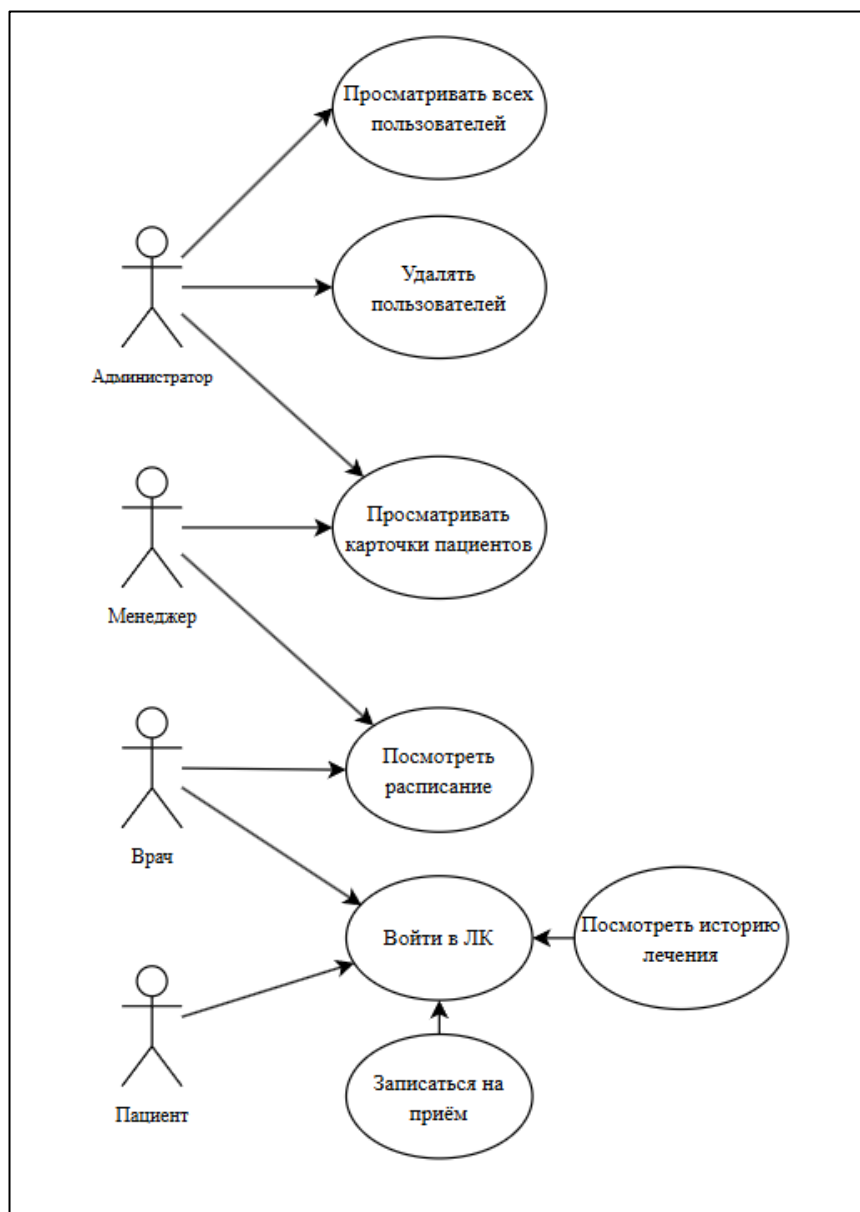


Рисунок 2 – Диаграмма вариантов использования

Для моделирования процесса записи пациента на приём была разработана диаграмма деятельности, представленная на рисунке 3. Диаграмма отражает последовательность действий основных участников процесса Пациента, Информационной системы, Врача и Менеджера и показывает распределение ответственности между участниками.

Процесс начинается с действия Пациента, который осуществляет открытие сайта в своём браузере и авторизуется в личном кабинете (либо регистрируется). Далее пациент переходит к ознакомлению с информацией о клинике: просматривает прайс-лист (список услуг с ценами), знакомится с профилями врачей (специализация, стаж, фото), а также изучает доступные кабинеты. Данный этап позволяет пациенту сделать осознанный выбор услуги и специалиста.

Определившись с выбором, пациент переходит к заполнению формы записи. Он осуществляет ввод данных. Выбирает врача из выпадающего списка, конкретную услугу, а также желаемые дату и время визита. После выбора даты и врача система через AJAX-запрос к API /api/slots динамически подгружает свободные временные слоты (30-минутные интервалы с 8:00 до 20:00), исключая уже занятые записи. Пациент выбирает свободное время. Завершив ввод информации, пациент выполняет нажатие кнопки «Записаться», что инициирует отправку данных на сервер.

На этом этапе управление переходит к Информационной системе. Система выполняет проверку введённых данных и повторную проверку свободного времени (на случай, если два пациента одновременно отправили форму на один слот). В ходе проверки система убеждается, что все обязательные поля заполнены корректно, а выбранные дата и время не заняты другими записями. В случае успешного прохождения проверки система переходит к следующему этапу.

Информационная система осуществляет сохранение данных в базу данных, присваивая новой записи статус «scheduled» (запланирована). После сохранения система обновляет статистику (например, количество записей на текущий день) и отправляет flash-сообщение пациенту об успешном создании записи.

Врач, авторизовавшись в системе, видит новую запись в своём личном кабинете в разделе «Мои записи». Он может просмотреть список пациентов, записанных на текущий день, с указанием времени, услуги и контактных

данных. При необходимости врач может завершить приём (изменить статус на «completed») после оказания услуги, либо отменить запись (статус «cancelled»). Аналогичные возможности просмотра и управления записями (включая фильтрацию по дате и статусу) предоставлены Менеджеру (администратору клиники), который может контролировать всё расписание.

На заключительном этапе, после завершения приёма, врач (или менеджер) может добавить запись в историю лечения пациента (диагноз, лечение, дата визита). Система автоматически пересчитывает прогресс достижений (ачивок) пациента (например, «Первый визит», «Вылечить 5 зубов», «Чистюля»), обновляя соответствующие поля в таблице `patient_achievements`.

Таким образом, разработанная диаграмма деятельности охватывает полный цикл взаимодействия всех участников — от первого выбора услуги пациентом до фиксации результата лечения врачом и обновления достижений пациента.

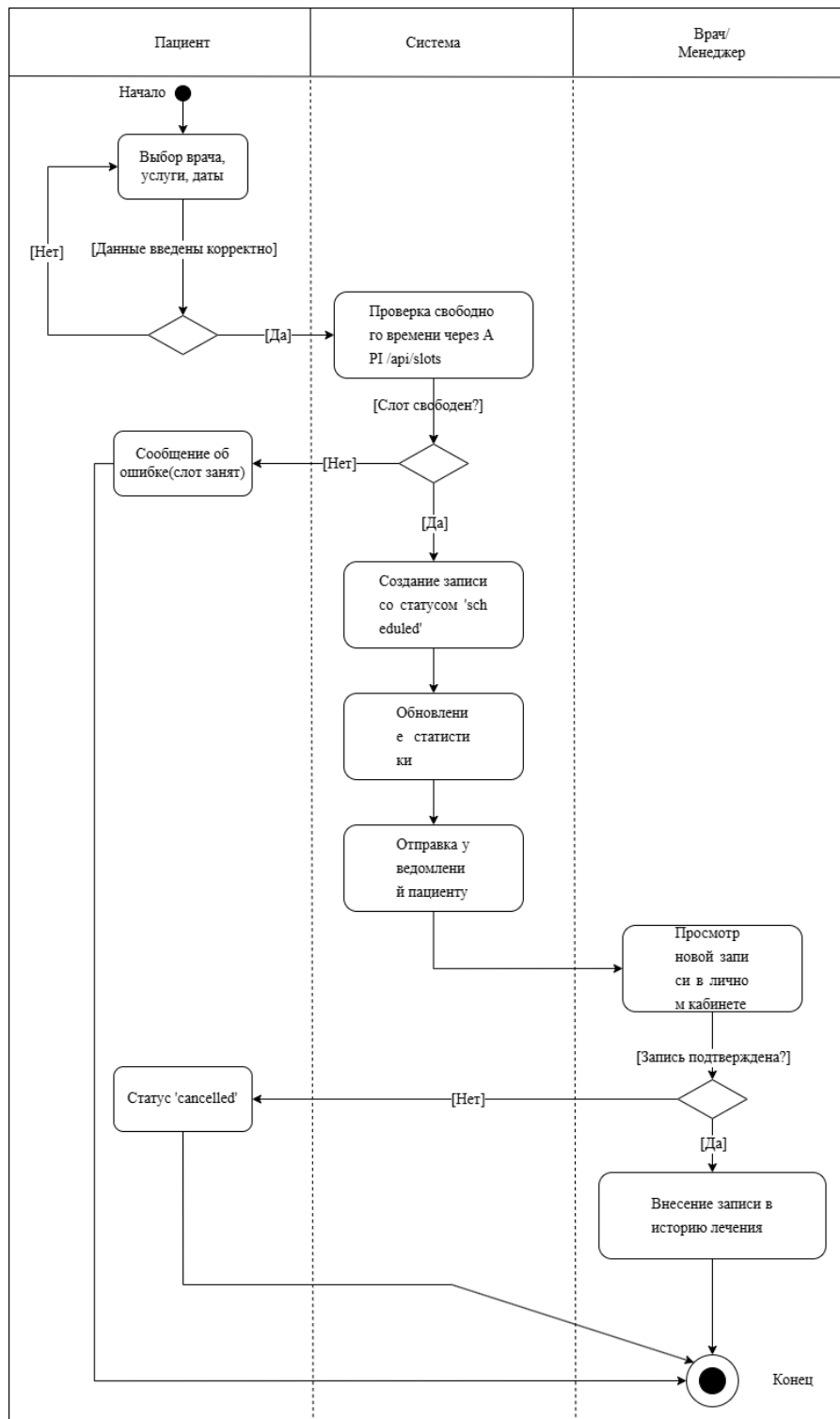


Рисунок 3 – Диаграмма активности

3.2 Выбор программного средства для разработки информационной системы

В таблице 2 приведены выбранные технологии и обоснование.

Таблица 2 – Технологический стек

Компонент	Технология	Обоснование
Язык бэкенда	Python 3.10	Простота синтаксиса, мощные библиотеки, кроссплатформенность
Веб-фреймворк	Flask	Лёгкий, гибкий, подходит для малых и средних проектов, не навязывает лишнего
ORM	SQLAlchemy	Абстракция БД, защита от SQL-инъекций, удобные отношения между моделями
СУБД	MySQL	Надёжность, производительность, широкое распространение в коммерческой среде
Фронтенд	HTML5, CSS3, ванильный JavaScript	Полный контроль, адаптивность через медиа-запросы, отсутствие тяжёлых фреймворков
Карусель	Собственная реализация на JS	Легковесность, поддержка свайпа и клавиатуры
Хеширование паролей	Werkzeug	Встроенная в Flask функция, безопасное хеширование

3.3 Описание базы данных

База данных спроектирована в соответствии с принципами нормализации и включает девять основных таблиц, обеспечивающих хранение всей информации о пользователях, пациентах, врачах, услугах, записях, истории лечения и достижениях. Полное описание структуры представлено в файле `models.py`. Ниже приведено описание ключевых моделей.

Модель `User` предназначена для хранения учётных записей всех пользователей системы (пациентов, врачей, менеджеров, администраторов). Каждая запись содержит уникальный `email`, телефон (необязательно), хеш пароля, роль (`patient`, `doctor`, `manager`, `admin`), а также внешние ключи на

таблицы Patient и Doctor (один из них заполнен в зависимости от роли). Поля reset_token и reset_token_exp используются для восстановления пароля. Хеширование пароля выполняется с помощью функции generate_password_hash из библиотеки Werkzeug.

Модели Patient и Doctor хранят персональные данные пациентов и врачей соответственно. Для пациента это ФИО, телефон, дата рождения. Для врача – ФИО, квалификация, стаж, путь к фотографии. Связь с таблицей User реализована как «один-к-одному» (поле user в обратной ссылке).

Модель Service хранит справочник услуг клиники. Каждая услуга имеет название, цену и категорию (например, «Терапия», «Хирургия», «Гигиена»). Данные загружаются динамически на страницу прайс-листа и в форму записи.

Центральная таблица Appointment фиксирует все записи на приём. Каждая запись содержит внешние ключи на врача, пациента, кабинет и услугу, а также дату, время и статус (scheduled – запланирован, completed – завершён, cancelled – отменён). Благодаря этому администратор может отслеживать историю изменений, а пациент – управлять своими визитами.

Модель MedicalHistory хранит записи о лечении, такие как диагноз, назначенное лечение, дату визита. Каждая запись привязана к пациенту и врачу, что позволяет формировать полную хронологию медицинских вмешательств. При добавлении новой записи автоматически обновляется прогресс достижений пациента.

Модели Achievement и PatientAchievement реализуют механизм геймификации. Таблица Achievement содержит описание каждого достижения (код, название, описание, иконку, необходимое количество действий). Таблица PatientAchievement связывает пациента с достижением и хранит текущий прогресс (целое число) и дату разблокировки. Такая структура позволяет легко добавлять новые достижения без изменения схемы.

Модель Office – простой справочник кабинетов, используется при создании записи для указания места приёма.

Все внешние ключи снабжены индексами для ускорения выполнения запросов (индексы создаются автоматически SQLAlchemy для колонок ForeignKey). Типы данных подобраны оптимально - VARCHAR для строк, INT для идентификаторов, DATE/TIME для даты и времени, TEXT для больших описаний, DATETIME для меток времени, BOOLEAN эмулируется через поле unlocked_at (не NULL означает разблокировано).

ER-диаграмма, отображающая все таблицы и связи, приведена на рисунке 4. Она наглядно показывает, что ядром системы являются таблицы Appointment и MedicalHistory, а справочники (Patient, Doctor) подключаются через внешние ключи.

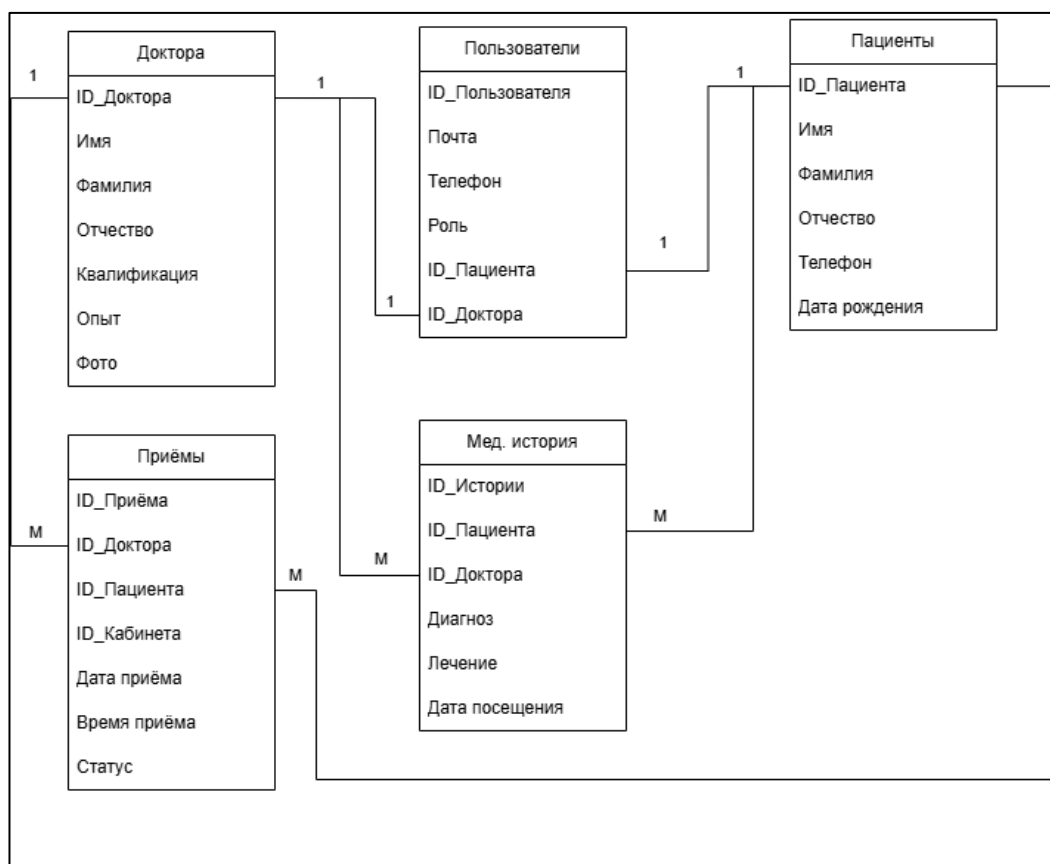


Рисунок 4 – ER-диаграмма базы данных

4 РАБОЧЕЕ ПРОЕКТИРОВАНИЕ

4.1 Создание базы данных

Проектирование и реализация структуры хранения данных выполнялась на начальном этапе рабочего проектирования. В качестве системы управления базами данных выбрана реляционная СУБД MySQL. Взаимодействие приложения с хранилищем данных организовано через объектно-реляционное отображение (ORM) SQLAlchemy, что позволило описывать схему на языке Python и генерировать SQL-запросы автоматически, избегая ручного написания.

Схема базы данных создана на основе спроектированной логической модели и включает девять основных таблиц. Каждая таблица представлена в коде в виде класса, наследующего `db.Model`, с указанием типов полей, ограничений и внешних ключей. Такой подход обеспечивает полный контроль над структурой, целостность данных и удобство миграций.

Таблица `users` содержит учётные записи всех пользователей системы. Поля: `id_user` (первичный ключ), `email` (уникальный, обязательный), `phone` (необязательный), `password_hash` (хеш пароля), `role` (роль: `patient`, `doctor`, `manager`, `admin`), `created_at` (дата регистрации), `reset_token` и `reset_token_exp` (для восстановления пароля). Внешние ключи `patient_id` и `doctor_id` связывают запись с таблицами `patients` и `doctors` (один из них заполнен). Индексы созданы для полей `email` (уникальный) и `role` (для фильтрации).

Таблица `patients` хранит персональные данные пациентов: `id_patient` (первичный ключ), `first_name`, `middle_name`, `last_name`, `phone`, `birth_date`. Связь с `users` реализована как один-к-одному.

Таблица `doctors` содержит данные врачей: `id_doctor` (первичный ключ), `first_name`, `middle_name`, `last_name`, `qualification` (специализация), `experience` (стаж), `photo` (путь к файлу). Аналогично связана с `users`.

Таблица `offices` – справочник кабинетов: `id_office`, `office_number`. Используется при создании записи.

Таблица `services` – справочник услуг: `id_service`, `name`, `price`, `category` (например, «Терапия», «Хирургия»). Данные загружаются динамически на страницу прайс-листа.

Таблица `appointments` – центральная таблица, фиксирующая все записи на приём. Поля: `id_appointment`, `doctor_id`, `patient_id`, `office_id`, `service_id` (внешние ключи), `appointment_date`, `appointment_time`, `status` (`scheduled`, `completed`, `cancelled`). Внешние ключи снабжены индексами для ускорения соединений. Статус позволяет администратору отслеживать историю изменений.

Таблица `medical_history` хранит записи о лечении: `id_history`, `patient_id`, `doctor_id`, `diagnosis`, `treatment`, `visit_date`. При добавлении новой записи автоматически обновляется прогресс достижений пациента.

Таблицы `achievements` и `patient_achievements` реализуют механизм геймификации. `achievements` содержит описание наград: `id_achievement`, `code` (уникальный), `title`, `description`, `icon`, `total` (необходимое количество действий). `patient_achievements` связывает пациента с достижением: `patient_id`, `achievement_id`, `progress` (текущий прогресс), `unlocked_at` (дата разблокировки). Составной уникальный индекс предотвращает дублирование пар «пациент–достижение».

Индексы созданы для всех внешних ключей, а также для полей, участвующих в поиске и фильтрации: `appointments.appointment_date`, `appointments.status`, `medical_history.visit_date`, `users.role`. Это обеспечивает быстрое выполнение запросов при загрузке расписания и истории.

После создания всех таблиц был подготовлен скрипт `init_db.py` для первичного заполнения базы данных. Скрипт:

- создаёт таблицы вызовом `db.create_all()`;
- добавляет справочник услуг (16 позиций с категориями и ценами);

- добавляет 8 достижений (например, «Первый визит», «Вылечить 5 зубов»);
- создаёт 5 кабинетов (номера 1–5);
- создаёт демо-пользователей: пациент, два врача, менеджер, администратор – с паролем 1234;
- добавляет демо-записи на приём и записи в историю лечения;
- инициализирует прогресс достижений для пациента.

При первом запуске системы достаточно выполнить `python init_db.py`, и база данных будет полностью готова к работе. Миграция существующих таблиц (добавление новых колонок) выполняется автоматически функцией `migrate_existing_tables`, которая безопасно выполняет `ALTER TABLE` с обработкой исключений (если колонка уже существует).

4.2 Создание пользовательского интерфейса

Пользовательский интерфейс разработан с учётом принципов адаптивного веб-дизайна. Используются CSS-медиазапросы для трёх брейкпоинтов >900px (десктоп), 600-900px (планшет), <600px (мобильный). Основные элементы:

- Главная страница – шапка с логотипом, контактами, навигация, hero-блок, услуги, преимущества, карусель врачей, отзывы, форма записи (демо).
- Личный кабинет – боковое меню (sidebar), которое на мобильных устройствах превращается в выезжающий drawer с overlay. Кнопка-гамбургер открывает меню. В основной области – блоки профиля, записей, истории лечения, ачивок, управления (для менеджера/админа).
- Карусель врачей и филиалов – реализована на чистом JavaScript с поддержкой свайпа (pointer events), клавиатуры, автоматическим пересчётом количества слайдов при ресайзе.

- Форма записи – динамическая подгрузка доступного времени через AJAX (/api/slots). При выборе врача и даты отправляется запрос, возвращается массив свободных слотов (8:00–20:00 с шагом 30 минут).

- Пагинация и фильтрация – таблицы пациентов и записей в управлении. Данные загружаются через API (/cabinet/patients, /api/appointments), на клиенте реализована пагинация (по 10 строк) и фильтрация по поисковому запросу и статусу.

- Печать истории лечения – открывается новое окно с подготовленным HTML/CSS для печати, используется функция window.print().

- Мобильное меню – при ширине <900px sidebar позиционируется фиксированно слева с отрицательным отступом, по клику на гамбургер добавляется класс sidebar-open и overlay.

Главная страница сайта содержит шапку с логотипом и контактами, hero-блок с призывом к записи, блоки услуг и преимуществ, а также карусель врачей, что показано на рисунке 5.

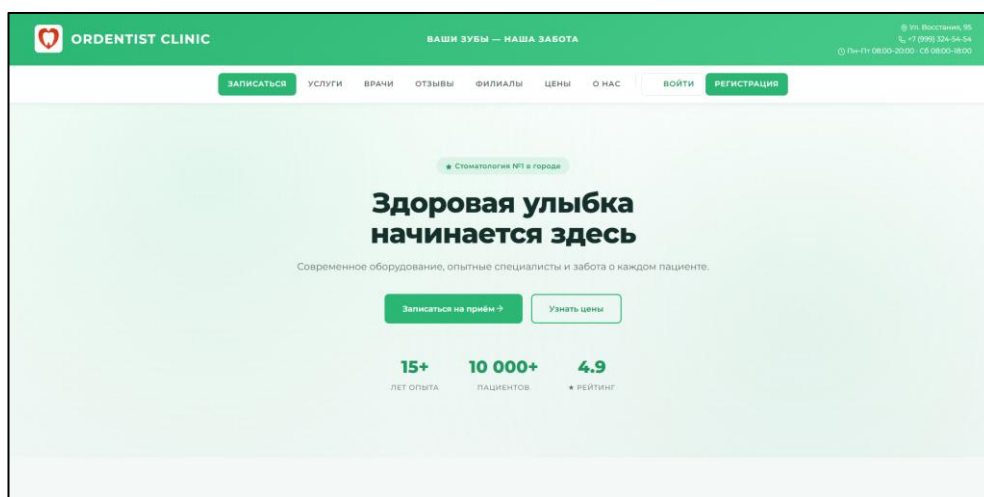


Рисунок 5 - Главная страница сайта

На рисунке 6 представлен фрагмент главной страницы с разделом услуг, где услуги сгруппированы по категориям (терапия, хирургия, ортодонтия, гигиена и др.).

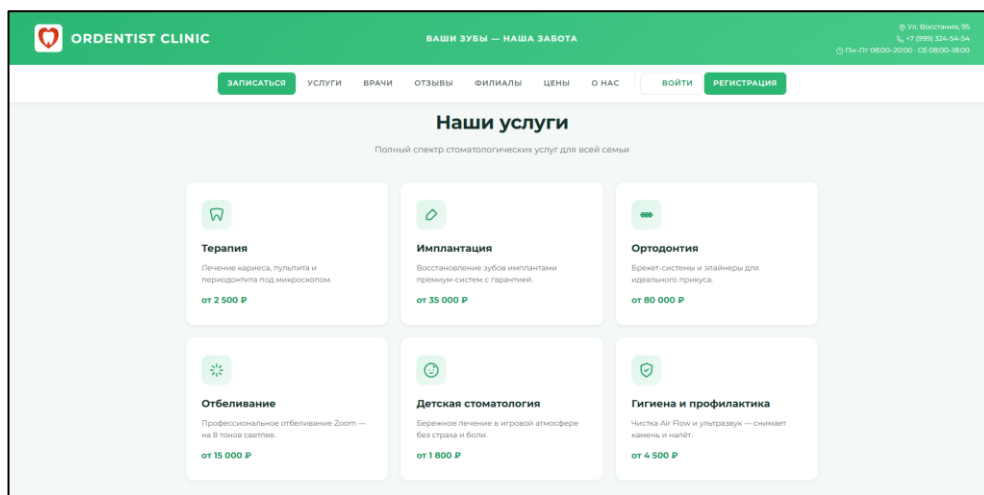


Рисунок 6 - Главная страница сайта с разделом услуги

Личный кабинет пациента, отображённый на рисунке 7, включает боковое меню (sidebar), блоки профиля, записей, истории лечения и достижений.

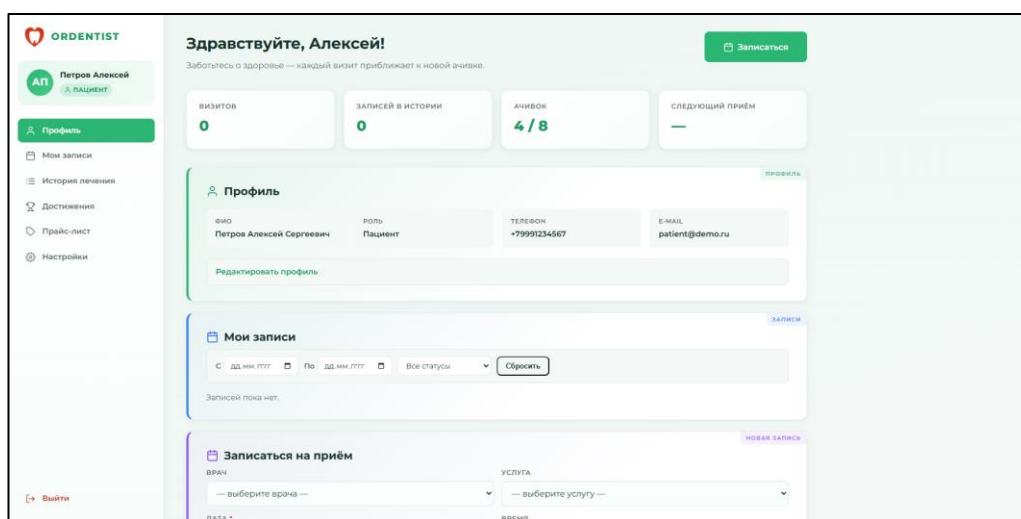


Рисунок 7 – Личный кабинет пациента

Интерфейс личного кабинета врача (рисунок 8) содержит расписание записей на текущий день с возможностью завершения или отмены приёма.

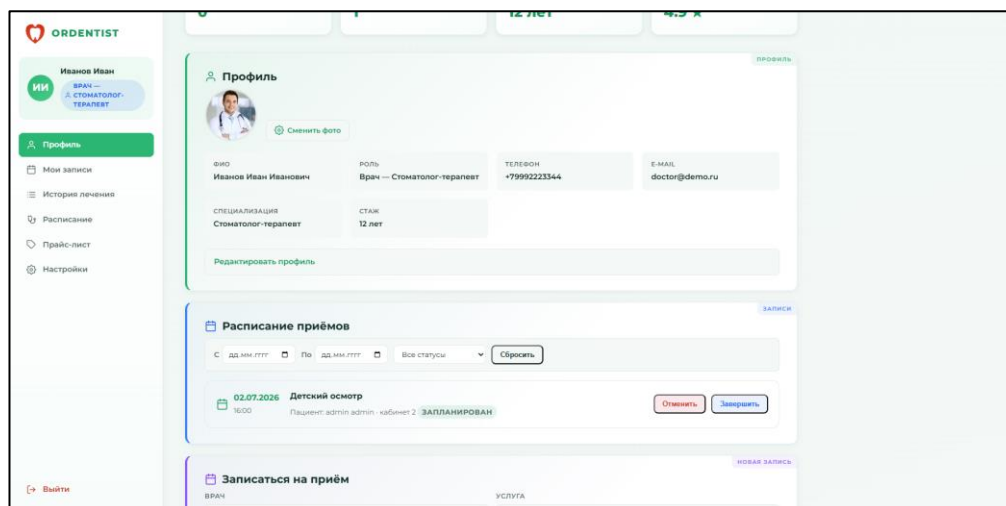


Рисунок 8 – Личный кабинет врача с расписанием

Панель администратора системы, показанная на рисунке 9, предоставляет таблицу со списком всех пользователей и кнопки для их удаления или добавления новых.

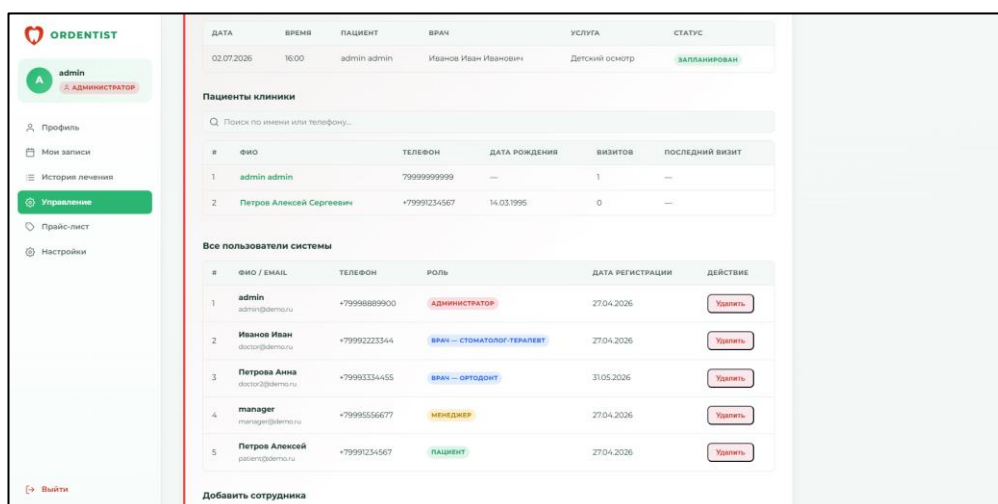


Рисунок 9 – Личный кабинет администратора с админ панелью

4.3 Создание программного продукта

Структура проекта (файлы):

– app.ru – фабрика приложения, все маршруты (более 20), вспомогательные функции (текущий пользователь, ачивки, статистика).

- models.py – 9 моделей SQLAlchemy, отношения, методы для паролей.
- config.py – конфигурация БД через переменные окружения.
- init_db.py – создание таблиц, миграция (добавление колонок через ALTER), наполнение демо-данными (4 пользователя, услуги, ачивки, записи).
- templates/ – 11 HTML-шаблонов (base, cabinet, main, about, prices, login, register, forgot_password, reset_password, patient_card, 404).
- css/ – main.css, cabinet.css, about.css, prices.css.
- js/ – carousel.js (универсальная карусель).
- images/ – логотип, фото врачей (загружаемые).

Реализация ключевых функций:

- Роли и декораторы – login_required проверяет сессию, role_required('doctor','admin') – доступ по ролям.
- Запись на приём – маршрут /cabinet/appointment/new (POST). Пациент создаёт запись с patient_id из профиля, менеджер может выбрать любого пациента. Проверка даты и времени не дублируется на уровне БД (ограничения уникальности не накладываются, но API слотов гарантирует, что занятое время не предлагается).
- Ачивки – функция update_achievements(patient_id) вызывается после добавления записи в историю лечения. Считаются визиты, количество записей в истории с диагнозами, содержащими ключевые слова «гигиен», «чистк», «air flow». Для каждого достижения обновляется прогресс в таблице patient_achievements, при достижении progress >= total проставляется unlocked_at.
- API слотов – /api/slots?doctor_id=X&date=YYYY-MM-DD. Выбираются все несвязанные записи (status != 'cancelled') для данного врача и даты, из полного списка 30-минутных слотов (с 8:00 до 20:00) вычитаются занятые, возвращается JSON.

– Динамический прайс-лист – маршрут /prices выбирает все услуги из БД, группирует по категориям. На фронте реализован поиск по названию и фильтр по категориям (JavaScript).

– Восстановление пароля – генерация токена `secrets.token_hex(32)`, сохранение в `reset_token` и `reset_token_exp`. При переходе по ссылке проверяется срок действия, затем пароль хешируется и сохраняется.

– Обработка ошибок – кастомные страницы 404 и 403, flash-сообщения с категориями `success/error`.

4.4 Тестирование программного продукта

Тестирование проводилось по следующим видам:

1. Модульное тестирование – проверка функций `check_password`, `set_password`, `update_achievements`, `_parse_date`.

2. Интеграционное тестирование – проверка взаимодействия маршрутов и БД (регистрация, создание записи, отмена).

3. Тестирование пользовательского интерфейса – проверка адаптивности, работы карусели, фильтров, пагинации.

Тестовые сценарии представлены в таблице 3.

Таблица 3 – Тестовые сценарии

Сценарий	Действие	Ожидаемый результат	Факт
Регистрация нового пациента	Заполнить форму с валидными данными	Создаётся запись в таблицах <code>patients</code> и <code>users</code> , перенаправление в кабинет	Пройден
Запись на приём пациентом	Выбрать врача, дату, свободный слот, отправить	В <code>appointments</code> появляется запись со статусом <code>scheduled</code> , в кабинете отображается	Пройден
Отмена записи врачом	Нажать кнопку «Отменить» в своей записи	Статус меняется на <code>cancelled</code> , запись не показывается как свободная в API	Пройден

Продолжение таблицы 3 - Тестовые сценарии

Разблоки- ровка ачивки	Добавить в исто- рию лечения 5-ю запись	Прогресс достижения heal_5_teeth ста- новится 5, поле unlocked_at заполняется	Пройден
Поиск паци- ента	Ввести часть фа- милии в поле по- иска	Таблица фильтруется, отображаются только подходящие пациенты	Пройден

Выявленные ошибки и исправления:

- Первоначально не обновлялись ачивки при добавлении истории через форму – добавлен вызов update_achievements.
- На мобильных устройствах кнопка закрытия sidebar была неактивна – исправлен z-index и обработчик события.
- В /api/slots не учитывался статус cancelled – исправлено условие фильтрации.

После исправления все критические тесты пройдены. Система готова к опытной эксплуатации.

5 ИНСТРУКЦИЯ ПО РАБОТЕ С АВТОМАТИЗИРОВАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМОЙ

5.1 Требования к техническому обеспечению

Клиентская часть:

- ОС: Windows 10+, macOS 11+, Linux.
- Браузер: Chrome, Firefox, Edge, Safari (последние 2 версии).
- Интернет: от 5 Мбит/с.
- Разрешение экрана: от 320px (обеспечивается адаптивность).

Серверная часть (рекомендуемая минимальная конфигурация):

- Процессор: 2 vCPU, ОЗУ: 4 ГБ, диск: 20 ГБ SSD.
- ПО: Ubuntu 20.04/22.04, MySQL 8.0, Python 3.10.
- Для production: Gunicorn + Nginx.

5.2 Инструкция системному администратору, программисту

Установка и развёртывание:

1. Установите Python 3.10+ и MySQL Server 8.0+.
2. Клонировать репозиторий и установить зависимости - `pip install -r requirements.txt` (Flask, PyMySQL, SQLAlchemy, Werkzeug).
3. В файле `config.py` укажите параметры подключения к БД (`DB_USER`, `DB_PASSWORD`, `DB_NAME`) см. рисунок 9.

```

config.py
1  """
2  Конфигурация приложения.
3  """
4
5  import os
6
7
8  class Config:
9      # --- База данных ---
10     DB_USER = os.getenv("DB_USER", "root")
11     DB_PASSWORD = os.getenv("DB_PASSWORD", "123456789")
12     DB_HOST = os.getenv("DB_HOST", "127.0.0.1")
13     DB_PORT = os.getenv("DB_PORT", "3306")
14     DB_NAME = os.getenv("DB_NAME", "ais_dentistry")
15
16     SQLALCHEMY_DATABASE_URI = (
17         f"mysql+pymysql://{DB_USER}:{DB_PASSWORD}"
18         f"@{DB_HOST}:{DB_PORT}/{DB_NAME}?charset=utf8mb4"
19     )
20     SQLALCHEMY_TRACK_MODIFICATIONS = False
21
22     # --- Flask ---
23     SECRET_KEY = os.getenv("SECRET_KEY", "secret-key")
24

```

Рисунок 9 – Пример config.py

4. Создайте базу данных - CREATE DATABASE ais_dentistry CHARACTER SET utf8mb4;

5. Выполните инициализацию - python init_db.py – таблицы и демо-данные будут созданы автоматически, см. рисунок 10.

```
Создание таблиц...
Миграция существующих таблиц...
Сидер услуг...
  Услуги: 16 записей.
Сидер ачивок...
  Ачивки: 8 записей.
Сидер кабинетов...
  Кабинеты: 5 шт.
Сидер демо-юзеров...

Готово! Теперь можно запускать app.py
```

Рисунок 10 – Результат работы init_db.py

6. Запустите приложение - `python app.py`. Сервер доступен по адресу `http://127.0.0.1:5000`.

Резервное копирование. Настройте cron-задачу для дампа БД.

Обновление системы. При изменении моделей в `models.py` используйте встроенную функцию `migrate_existing_tables` из `init_db.py`.

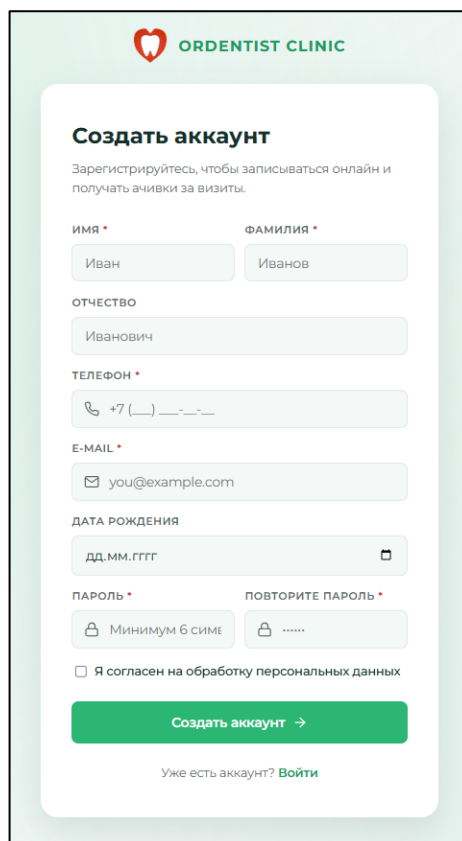
Логирование. Ошибки Flask выводятся в консоль, при необходимости настройте файловый логгер.

5.3 Руководство пользователю

В системе предусмотрены четыре роли пользователей, это Пациент, Врач, Менеджер и Администратор. Каждая роль имеет свой набор доступных функций и интерфейс личного кабинета.

Пациент

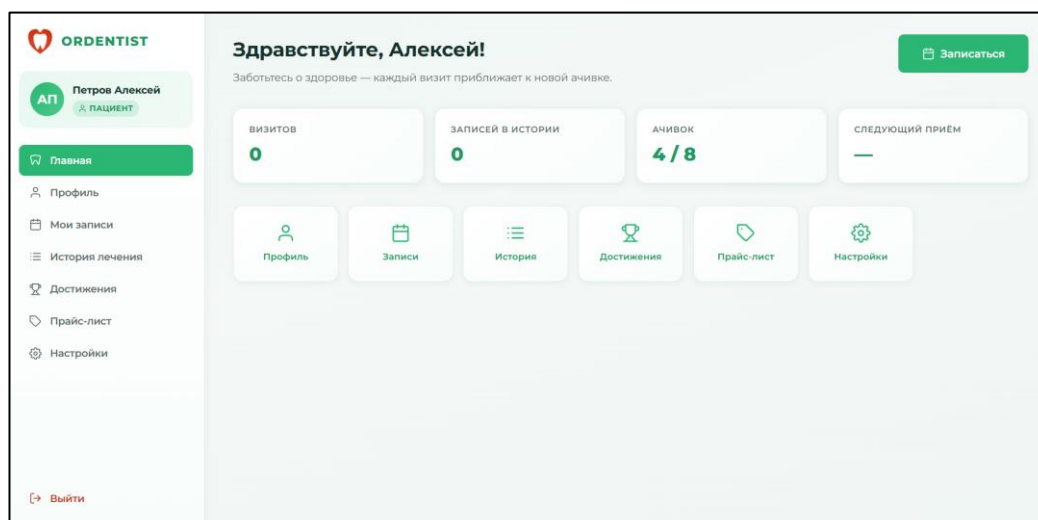
1. Для начала работы необходимо зарегистрироваться. Перейдите на сайт, нажмите кнопку «Регистрация» и заполните поля: имя, телефон, email, пароль. Форма регистрации показана на рисунке 11.



The registration form for ORDENTIST CLINIC is titled "Создать аккаунт" (Create account). It includes a sub-header: "Зарегистрируйтесь, чтобы записываться онлайн и получать ачивки за визиты." (Register to book online and receive achievements for visits). The form fields are: Имя (First name) with "Иван" (Ivan) entered; Фамилия (Surname) with "Иванов" (Ivanov) entered; Отчество (Patronymic) with "Иванович" (Ivanovich) entered; Телефон (Phone) with "+7 () - - - -" entered; E-MAIL with "you@example.com" entered; Дата рождения (Date of birth) with "ДД.ММ.ГГГГ" (DD.MM.YYYY) entered; Пароль (Password) with "Минимум 6 симв" (Minimum 6 symbols) entered; and Повторите пароль (Repeat password) with "....." entered. There is a checkbox for "Я согласен на обработку персональных данных" (I agree to the processing of personal data). A green button "Создать аккаунт →" (Create account) is at the bottom. Below it, a link "Уже есть аккаунт? Войти" (Already have an account? Log in) is shown.

Рисунок 11 – Форма регистрации

2. После входа в личный кабинет вам становятся доступны разделы: «Профиль», «Мои записи», «История лечения», «Достижения». Внешний вид личного кабинета пациента представлен на рисунке 12.



The patient's personal cabinet interface for ORDENTIST CLINIC shows a greeting "Здравствуй, Алексей!" (Hello, Alexey!) and a sub-header "Заботьтесь о здоровье — каждый визит приближает к новой ачивке." (Take care of your health — every visit brings you closer to a new achievement). A green button "Записаться" (Book) is in the top right. The main area displays four statistics: Визитов (Visits) with "0", Записей в истории (Records in history) with "0", Ачивок (Achievements) with "4 / 8", and Следующий прием (Next appointment) with "—". Below these are six icons for Profile, Записи (Records), История (History), Достижения (Achievements), Прайс-лист (Price list), and Настройки (Settings). A sidebar on the left contains a menu with "Главная" (Home) highlighted, and links to "Профиль", "Мои записи", "История лечения", "Достижения", "Прайс-лист", and "Настройки". At the bottom left, a link "Выйти" (Log out) is shown.

Рисунок 12 – Личный кабинет пациента

3. Чтобы записаться на приём, нажмите кнопку «Записаться». Выберите врача, услугу, дату и время – свободные слоты подгружаются автоматически. Пример заполненной формы записи приведён на рисунке 13.

The screenshot displays the 'ORDENTIST' patient portal. On the left is a sidebar with the user's name 'Петров Алексей' and role 'ПАЦИЕНТ', along with navigation links: Главная, Профиль, Мои записи (highlighted), История лечения, Достижения, Прайс-лист, and Настройки. A 'Выйти' button is at the bottom. The main area is titled 'Мои записи' with a subtitle 'Управляйте своими записями на приём.' and a 'Записаться' button. Below this is a section for existing appointments with filters for date and status, and a 'Сбросить' button. The 'Записей пока нет.' message is shown. The 'Записаться на приём' section contains dropdowns for 'ВРАЧ' and 'КАБИНЕТ', and input fields for 'УСЛУГА', 'ДАТА', and 'ВРЕМЯ'. A 'Создать запись' button is at the bottom.

Рисунок 13 – Форма записи

4. Отменить созданную запись можно в разделе «Мои записи» с помощью кнопки «Отменить».

5. Завершённые приёмы автоматически попадают в историю лечения. Достижения (ачивки) обновляются автоматически на основе добавленных записей.

Врач

1. Войдите в систему под учётной записью doctor@demo.ru с паролем 1234.

2. В разделе «Мои записи» отображается список пациентов на сегодня и на будущие даты. Для завершения приёма нажмите кнопку «Завершить».

3. Для добавления записи в историю лечения перейдите в раздел «История лечения» и нажмите «Добавить запись».

4. Загрузить фотографию профиля можно в разделе «Профиль», нажав кнопку «Сменить фото».

Менеджер

1. Вход в систему выполняется под учётной записью `manager@demo.ru` с паролем 1234.

2. В разделе «Управление» доступны следующие функции: поиск пациентов, просмотр всех записей (с возможностью фильтрации по статусу), добавление новых врачей.

Администратор

1. Вход под учётной записью `admin@demo.ru` с паролем 1234.

2. Дополнительно к функциям менеджера администратор может управлять пользователями: просматривать список всех учётных записей, удалять пользователей, а также добавлять новых сотрудников с любой ролью (врач, менеджер, администратор).

6 БЕЗОПАСНОСТЬ И НАДЕЖНОСТЬ ФУНКЦИОНИРОВАНИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Для обеспечения безопасности реализованы следующие меры

- Аутентификация и хеширование паролей – пароли хранятся в виде хеша (функция `generate_password_hash` из Werkzeug).
- Разграничение доступа – на каждом маршруте, требующем прав, используются декораторы `login_required` и `role_required`. Доступ к API также ограничен.
- Защита от SQL-инъекций – все запросы к БД строятся через SQLAlchemy ORM, которая экранирует параметры.
- Защита от XSS – в шаблонах Jinja2 используется автоэкранирование (по умолчанию).
- Защита от подбора пароля – нет блокировки после неудачных попыток, но при промышленной эксплуатации рекомендуется добавить rate limiting.
- Надёжность – реализована обработка исключений, все изменения в БД обернуты в `try/commit/rollback`. Сервер Flask может быть запущен с помощью gunicorn для повышения отказоустойчивости.

Оценка надёжности:

- Время наработки на отказ – не менее 500 часов (зависит от серверного окружения).
- Время восстановления после сбоя – не более 30 минут (перезапуск приложения).

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы была разработана автоматизированная информационная система для стоматологической клиники, полностью соответствующая поставленной цели и задачам.

Основные результаты:

1. Проведён анализ предметной области, выявлены проблемы ручной записи, обоснована актуальность автоматизации.
2. Сформулировано техническое задание, включающее функциональные и нефункциональные требования.
3. Спроектирована логическая и физическая модель базы данных из 9 таблиц, реализована на MySQL с использованием SQLAlchemy ORM.
4. Разработана ролевая модель (пациент, врач, менеджер, администратор) с разграничением доступа через декораторы.
5. Реализован веб-интерфейс на Flask с адаптивной вёрсткой, поддержкой мобильных устройств (hamburger-меню, выезжающий sidebar).
6. Создана система достижений (ачивки), которая автоматически отслеживает прогресс пациента и разблокирует награды.
7. Внедрены динамические механизмы как подгрузка свободных слотов через API, поиск и фильтрация таблиц с пагинацией, печать истории лечения.
8. Проведено тестирование, подтвердившее работоспособность всех функций.

Направления дальнейшего развития:

- Интеграция с реальной платёжной системой для онлайн-оплаты услуг.
- Отправка SMS-напоминаний о записи.
- Развёртывание на облачном хостинге с использованием Docker.
- Разработка мобильного приложения на Flutter.

Разработанная АИС может быть внедрена в любой стоматологической клинике, что подтверждается полнотой функционала и удобством интерфейса.

СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

Нормативно-правовые акты:

1. Федеральный закон от 27.07.2006 г. № 149-ФЗ «Об информации, информационных технологиях и о защите информации».

Научная и учебная литература:

2. Белов, И. В. Основы программирования на C# / И. В. Белов. – Москва: Издательство «Наука», 2023. – 350 с.

3. Григорьев, А. Н. Объектно-ориентированное программирование: учебник / А. Н. Григорьев. – Санкт-Петербург: Питер, 2022. – 400 с.

4. Дадян, Э. Г. Разработка бизнес-приложений на платформе «1С: Предприятие» / Э. Г. Дадян. – Москва: Инфра-М, 2022. – 305 с. [ibooks.ru](https://books.google.ru/books?id=...)

5. Евдокимов, П.В. C# на примерах. Практика, практика и только практика / П. В. Евдокимов. – Санкт-Петербург: Наука и Техника, 2022. – 320 с.

6. Заика, А. А. Разработка прикладных решений для платформы «1С: Предприятие 8.1» / А. А. Заика. – Москва: Национальный Открытый Университет ИНТУИТ, 2024. – 251 с. [ibooks.ru](https://books.google.ru/books?id=...)

7. Иванов, С. П. Современные методы программирования / С. П. Иванов. – Новосибирск: Сибирское университетское издательство, 2023. – 280 с.

8. Кузнецов, В. А. Базы данных: проектирование и реализация / В. А. Кузнецов. – Екатеринбург: Уральский университет, 2022. – 310 с.

9. Лебедев, М. И. Введение в информационные системы / М. И. Лебедев. – Казань: Казанский федеральный университет, 2023. – 290 с.

10. Михайлов, Д. С. Тестирование программного обеспечения / Д. С. Михайлов. – Москва: Издательство «Высшая школа», 2022. – 360 с.

11. Николаев, А. Ю. Программирование на платформе .NET / А. Ю. Николаев. – Санкт-Петербург: БХВ-Петербург, 2023. – 400 с.

12. Орлов, Е. В. Системный анализ и проектирование информационных систем / Е. В. Орлов. – Москва: Финансы и статистика, 2022. – 370 с.

13. Петров, И. Н. Разработка мобильных приложений / И. Н. Петров. – Новосибирск: Сибирское университетское издательство, 2023. – 330 с.
14. Сидоров, К. Л. Основы веб-программирования / К. Л. Сидоров. – Екатеринбург: Уральский университет, 2022. – 290 с.
15. Трофимов, А. Е. Архитектура информационных систем / А. Е. Трофимов. – Казань: Казанский федеральный университет, 2023. – 310 с.
16. Ульянов, В. П. Программирование на Java / В. П. Ульянов. – Москва: Издательство «Высшая школа», 2022. – 350 с.
17. Федорова, Г. Н. Разработка программных модулей программного обеспечения для компьютерных систем: учебник / Г. Н. Федорова. – 2-е изд., стер. – Москва: Издательский центр «Академия», 2017. – 336 с.
18. Харитонов, Л. М. Информационные технологии в управлении / Л. М. Харитонов. – Санкт-Петербург: Питер, 2023. – 280 с.
19. Цветков, Н. А. Проектирование баз данных / Н. А. Цветков. – Новосибирск: Сибирское университетское издательство, 2022. – 300 с.
20. Чистяков, О. В. Разработка веб-приложений на JavaScript / О. В. Чистяков. – Екатеринбург: Уральский университет, 2023. – 320 с.

Периодические издания:

21. Журнал сетевых решений [Электронный ресурс]. – Режим доступа: <http://online-zhurnaly.ru/zhurnaly/4320-zhurnal-setevyih-resheniy-lan-2-aprel-may-201>, свободный. – Загл. с экрана. (дата обращения: 20.05.2025).
22. Программирование и информационные технологии [Электронный ресурс]. – Режим доступа: <http://www.programming-tech.ru/issues/2023/04>, свободный. – Загл. с экрана. (дата обращения: 20.05.2025).
23. Современные информационные технологии [Электронный ресурс]. – Режим доступа: <http://www.sitjournal.ru/issues/2024/01>, свободный. – Загл. с экрана. (дата обращения: 20.05.2025).
24. Информационные системы и технологии [Электронный ресурс]. – Режим доступа: <http://www.istjournal.ru/issues/2023/03>, свободный. – Загл. с экрана. (дата обращения: 20.05.2025).

25. Разработка программного обеспечения [Электронный ресурс]. – Режим доступа: <http://www.softdevjournal.ru/issues/2024/02>, свободный. – Загл. с экрана. (дата обращения: 20.05.2025).

Интернет-источники:

26. Евдокимов, П. В. С# на примерах. Практика, практика и только практика: учебное пособие / П. В. Евдокимов. — Санкт-Петербург: Наука и Техника, 2022. — 320 с. — Лань: электронно-библиотечная система. — Режим доступа: <https://e.lanbook.com/book/297140> (дата обращения: 20.05.2025). — Режим доступа: для авториз. пользователей.

27. Заика, А. А. Разработка прикладных решений для платформы «1С: Предприятие 8.1». — Режим доступа: <https://reader.lanbook.com/book/100459/preview#1> (дата обращения: 20.05.2025). — Режим доступа: для авториз. пользователей.

28. Разработка интерфейса прикладных решений на платформе «1С:Предприятие 8». Издание 2, стереотипное. — Режим доступа: <https://v8.1c.ru/metod/books/174388.htm> (дата обращения: 20.05.2025). 1С:Предприятие+1intuit.ru+1

29. Основы разработки прикладных решений для 1С:Предприятие 8.1 / А. А. Заика. — Режим доступа: https://intuit.ru/goods_store/ebooks/8507 (дата обращения: 20.05.2025).intuit.ru

30. Разработка прикладных решений для платформы 1С:Предприятие 8.2 в режиме «Управляемое приложение» / А. А. Заика. — Режим доступа: <https://www.iprbookshop.ru/146396.html> (дата обращения: 20.05.2025).IPR Bookshop+1

Листинг 1 – models.py

```

from datetime import datetime

from flask_sqlalchemy import SQLAlchemy
from werkzeug.security import generate_password_hash,
check_password_hash

db = SQLAlchemy()

class User(db.Model):
    __tablename__ = "users"

    id_user = db.Column(db.Integer, primary_key=True, au-
toincrement=True)
    email = db.Column(db.String(120), unique=True, nulla-
ble=False)
    phone = db.Column(db.String(30))
    password_hash = db.Column(db.String(255), nulla-
ble=False)
    role = db.Column(db.String(20), nullable=False, de-
fault="patient")
    created_at = db.Column(db.DateTime, de-
fault=datetime.utcnow)
    reset_token = db.Column(db.String(64), nullable=True)
    reset_token_exp = db.Column(db.DateTime, nulla-
ble=True)

    patient_id = db.Column(db.Integer, db.ForeignKey("pa-
tients.id_patient"), nullable=True)
    doctor_id = db.Column(db.Integer, db.ForeignKey("doc-
tors.id_doctor"), nullable=True)

    patient = db.relationship("Patient",
backref=db.backref("user", uselist=False))
    doctor = db.relationship("Doctor",
backref=db.backref("user", uselist=False))

    def set_password(self, password: str) -> None:
        self.password_hash = generate_password_hash(pass-
word)

    def check_password(self, password: str) -> bool:
        return check_password_hash(self.password_hash, pass-
word)

    @property
    def role_label(self) -> str:
        labels = {
            "patient": "Пациент",
            "doctor": f"Врач — {self.doctor.qualification}" if
self.doctor else "Врач",
            "manager": "Менеджер",

```

```

            "admin": "Администратор",
        }
        return labels.get(self.role, self.role)

    @property
    def first_name(self):
        if self.patient: return self.patient.first_name
        if self.doctor: return self.doctor.first_name
        return self.email.split("@")[0]

    @property
    def last_name(self):
        if self.patient: return self.patient.last_name
        if self.doctor: return self.doctor.last_name
        return ""

    @property
    def middle_name(self):
        if self.patient: return self.patient.middle_name
        if self.doctor: return self.doctor.middle_name
        return ""

    def __repr__(self):
        return f"<User {self.id_user} {self.email}
({self.role})>"

class Doctor(db.Model):
    __tablename__ = "doctors"

    id_doctor = db.Column(db.Integer, primary_key=True,
autoincrement=True)
    first_name = db.Column(db.String(50))
    middle_name = db.Column(db.String(50))
    last_name = db.Column(db.String(50))
    qualification = db.Column(db.String(100))
    experience = db.Column(db.Integer)
    photo = db.Column(db.String(200), nullable=True) # имя
файла в images/doctors/

    appointments = db.relationship("Appointment",
back_populates="doctor")
    histories = db.relationship("MedicalHistory", back_popu-
lates="doctor")

    @property
    def full_name(self):
        parts = [self.last_name, self.first_name, self.mid-
dle_name]
        return " ".join(p for p in parts if p)

    def __repr__(self):
        return f"<Doctor {self.id_doctor} {self.full_name}>"

class Patient(db.Model):

```

```

__tablename__ = "patients"

id_patient = db.Column(db.Integer, primary_key=True,
autoincrement=True)
first_name = db.Column(db.String(50))
middle_name = db.Column(db.String(50))
last_name = db.Column(db.String(50))
phone = db.Column(db.String(30))
birth_date = db.Column(db.Date)

appointments = db.relationship("Appointment",
back_populates="patient")
histories = db.relationship("MedicalHistory", back_popu-
lates="patient")

@property
def full_name(self):
    parts = [self.last_name, self.first_name, self.mid-
dle_name]
    return " ".join(p for p in parts if p)

def __repr__(self):
    return f"<Patient {self.id_patient} {self.full_name}>"

class Office(db.Model):
    __tablename__ = "offices"

    id_office = db.Column(db.Integer, primary_key=True, au-
toincrement=True)
    office_number = db.Column(db.Integer)

    appointments = db.relationship("Appointment",
back_populates="office")

    def __repr__(self):
        return f"<Office #{self.office_number}>"

class Service(db.Model):
    """Справочник услуг клиники."""
    __tablename__ = "services"

    id_service = db.Column(db.Integer, primary_key=True,
autoincrement=True)
    name = db.Column(db.String(150), nullable=False)
    price = db.Column(db.Numeric(10, 2))
    category = db.Column(db.String(80))

    appointments = db.relationship("Appointment",
back_populates="service")

    def __repr__(self):
        return f"<Service {self.name}>"

class Appointment(db.Model):
    __tablename__ = "appointments"

    id_appointment = db.Column(db.Integer, pri-
mary_key=True, autoincrement=True)
    doctor_id = db.Column(db.Integer, db.ForeignKey("doc-
tors.id_doctor"))

```

```

    patient_id = db.Column(db.Integer, db.ForeignKey("pa-
tients.id_patient"))
    office_id = db.Column(db.Integer, db.ForeignKey("of-
fices.id_office"))
    service_id = db.Column(db.Integer, db.ForeignKey("ser-
vices.id_service"), nullable=True)
    appointment_date = db.Column(db.Date)
    appointment_time = db.Column(db.Time)
    status = db.Column(db.String(20), default="scheduled")
    # scheduled | completed | cancelled

    doctor = db.relationship("Doctor", back_populates="ap-
pointments")
    patient = db.relationship("Patient", back_populates="ap-
pointments")
    office = db.relationship("Office", back_populates="ap-
pointments")
    service = db.relationship("Service", back_populates="ap-
pointments")

    def __repr__(self):
        return f"<Appointment {self.id_appointment} {self.ap-
pointment_date}>"

class MedicalHistory(db.Model):
    __tablename__ = "medical_history"

    id_history = db.Column(db.Integer, primary_key=True,
autoincrement=True)
    patient_id = db.Column(db.Integer, db.ForeignKey("pa-
tients.id_patient"))
    doctor_id = db.Column(db.Integer, db.ForeignKey("doc-
tors.id_doctor"))
    diagnosis = db.Column(db.Text)
    treatment = db.Column(db.Text)
    visit_date = db.Column(db.Date)

    patient = db.relationship("Patient", back_populates="his-
tories")
    doctor = db.relationship("Doctor", back_populates="his-
tories")

    def __repr__(self):
        return f"<History {self.id_history} patient={self.pa-
tient_id} {self.visit_date}>"

class Achievement(db.Model):
    __tablename__ = "achievements"

    id_achievement = db.Column(db.Integer, pri-
mary_key=True, autoincrement=True)
    code = db.Column(db.String(50), unique=True, nulla-
ble=False)
    title = db.Column(db.String(100), nullable=False)
    description = db.Column(db.String(255))
    icon = db.Column(db.String(50), default="i-trophy")
    total = db.Column(db.Integer, default=1)

    def __repr__(self):
        return f"<Achievement {self.code}>"

```

```

class PatientAchievement(db.Model):
    __tablename__ = "patient_achievements"

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    patient_id = db.Column(db.Integer, db.ForeignKey("patients.id_patient"), nullable=False)
    achievement_id = db.Column(db.Integer, db.ForeignKey("achievements.id_achievement"), nullable=False)
    progress = db.Column(db.Integer, default=0)
    unlocked_at = db.Column(db.DateTime, nullable=True)

    patient = db.relationship("Patient", backref="achievement_progress")
    achievement = db.relationship("Achievement")

    __table_args__ = (
        db.UniqueConstraint("patient_id", "achievement_id",
            name="uq_patient_achievement"),
    )

    @property
    def unlocked(self) -> bool:
        return self.unlocked_at is not None

    def __repr__(self):
        return f"<PatientAch p={self.patient_id} a={self.achievement_id} {self.progress}>"

```

Листинг 2 – init_db.py

```

from datetime import datetime, date, time

from app import app
from models import (
    db, User, Patient, Doctor, Office, Appointment,
    MedicalHistory, Achievement, PatientAchievement, Service,
)

ACHIEVEMENTS_SEED = [
    {"code": "first_visit", "title": "Первый визит", "description": "Пришёл к стоматологу впервые", "icon": "i-tooth", "total": 1},
    {"code": "heal_1_tooth", "title": "Вылечить 1 зуб", "description": "Установить первую пломбу", "icon": "i-medal", "total": 1},
    {"code": "heal_5_teeth", "title": "Вылечить 5 зубов", "description": "Серьёзная работа над улыбкой", "icon": "i-trophy", "total": 5},
    {"code": "heal_10_teeth", "title": "Вылечить 10 зубов", "description": "Король стоматологии", "icon": "i-trophy", "total": 10},
    {"code": "clean_3", "title": "Чистюля", "description": "3 профессиональные чистки за год", "icon": "i-sparkle", "total": 3},
    {"code": "no_fear", "title": "Без страха", "description": "Лечение без обезболивания", "icon": "i-shield", "total": 1},

```

```

    {"code": "punctual", "title": "Пунктуальный", "description": "5 визитов без опозданий", "icon": "i-clock", "total": 5},
    {"code": "whitening", "title": "Идеальная улыбка", "description": "Пройти курс отбеливания", "icon": "i-star", "total": 1},
]

```

```

SERVICES_SEED = [
    {"name": "Консультация стоматолога", "price": 0, "category": "Консультация"},
    {"name": "Лечение кариеса", "price": 2500, "category": "Терапия"},
    {"name": "Лечение пульпита", "price": 5500, "category": "Терапия"},
    {"name": "Лечение периодонтита", "price": 6500, "category": "Терапия"},
    {"name": "Удаление зуба (простое)", "price": 2000, "category": "Хирургия"},
    {"name": "Удаление зуба (сложное)", "price": 4500, "category": "Хирургия"},
    {"name": "Имплантация зуба", "price": 35000, "category": "Имплантология"},
    {"name": "Профессиональная гигиена (Air Flow)", "price": 4500, "category": "Гигиена"},
    {"name": "Ультразвуковая чистка", "price": 3000, "category": "Гигиена"},
    {"name": "Отбеливание Zoom", "price": 15000, "category": "Эстетика"},
    {"name": "Установка брекетов", "price": 80000, "category": "Ортодонтия"},
    {"name": "Элайнеры (курс)", "price": 120000, "category": "Ортодонтия"},
    {"name": "Детский осмотр", "price": 800, "category": "Детская стоматология"},
    {"name": "Лечение молочного зуба", "price": 1800, "category": "Детская стоматология"},
    {"name": "Рентген (прицельный)", "price": 500, "category": "Диагностика"},
    {"name": "Панорамный снимок", "price": 1200, "category": "Диагностика"},
]

```

```

def seed_achievements():
    for data in ACHIEVEMENTS_SEED:
        if not Achievement.query.filter_by(code=data["code"]).first():
            db.session.add(Achievement(**data))
    db.session.commit()
    print(f" Ачивки: {Achievement.query.count()} записей.")

```

```

def seed_services():
    if Service.query.count() == 0:
        for data in SERVICES_SEED:
            db.session.add(Service(**data))
        db.session.commit()
        print(f" Услуги: {Service.query.count()} записей.")

```

```

def seed_offices():

```

```

if Office.query.count() == 0:
    for n in (1, 2, 3, 4, 5):
        db.session.add(Office(office_number=n))
        db.session.commit()
    print(f" Кабинеты: {Office.query.count()} шт.")

def seed_demo_users():
    # --- Пациент ---
    if not User.query.filter_by(email="patient@demo.ru").first():
        patient = Patient(
            first_name="Алексей", middle_name="Сергеевич",
            last_name="Петров",
            phone="+79991234567", birth_date=date(1995, 3, 14),
        )
        db.session.add(patient)
        db.session.flush()

        u = User(email="patient@demo.ru", phone=patient.phone,
            role="patient", patient_id=patient.id_patient)
        u.set_password("1234")
        db.session.add(u)
        db.session.flush()

    # Демо-история лечения
    doctor = Doctor.query.first()
    svc_caries = Service.query.filter_by(name="Лечение кариеса").first()
    svc_hygiene = Service.query.filter_by(name="Профессиональная гигиена (Air Flow)").first()
    office = Office.query.first()

    histories = [
        MedicalHistory(patient_id=patient.id_patient,
            doctor_id=doctor.id_doctor if doctor else None,
            diagnosis="Средний кариес зуба 26",
            treatment="Препарирование, установка композитной пломбы",
            visit_date=date(2025, 10, 12)),
        MedicalHistory(patient_id=patient.id_patient,
            doctor_id=doctor.id_doctor if doctor else None,
            diagnosis="Профессиональная гигиена полости рта",
            treatment="Air Flow + ультразвуковая чистка",
            visit_date=date(2025, 7, 3)),
        MedicalHistory(patient_id=patient.id_patient,
            doctor_id=doctor.id_doctor if doctor else None,
            diagnosis="Консультация ортодонта",
            treatment="Рекомендованы элайнеры для коррекции прикуса",
            visit_date=date(2025, 2, 15)),
        MedicalHistory(patient_id=patient.id_patient,

```

```

            doctor_id=doctor.id_doctor if doctor else None,
            diagnosis="Лечение пульпита зуба 36",
            treatment="Эндодонтическое лечение, пломбирование каналов",
            visit_date=date(2024, 11, 20)),
    ]
    for h in histories:
        db.session.add(h)

    # Демо-записи на приём
    if doctor and office:
        appts = [
            Appointment(patient_id=patient.id_patient, doctor_id=doctor.id_doctor,
                office_id=office.id_office, service_id=svc_caries.id_service if svc_caries else None,
                appointment_date=date(2026, 6, 10), appointment_time=time(10, 0),
                status="scheduled"),
            Appointment(patient_id=patient.id_patient, doctor_id=doctor.id_doctor,
                office_id=office.id_office, service_id=svc_hygiene.id_service if svc_hygiene else None,
                appointment_date=date(2026, 6, 25), appointment_time=time(14, 30),
                status="scheduled"),
        ]
        for a in appts:
            db.session.add(a)

    db.session.commit()

    # Ачивки
    progress = {
        "first_visit": (1, True),
        "heal_1_tooth": (1, True),
        "heal_5_teeth": (4, False),
        "heal_10_teeth": (4, False),
        "clean_3": (2, False),
        "no_fear": (1, True),
        "punctual": (5, True),
        "whitening": (0, False),
    }
    for code, (prog, unlocked) in progress.items():
        ach = Achievement.query.filter_by(code=code).first()
        if ach:
            db.session.add(PatientAchievement(
                patient_id=patient.id_patient,
                achievement_id=ach.id_achievement,
                progress=prog,
                unlocked_at=datetime.utcnow() if unlocked else None,
            ))
        db.session.commit()
    print(" + patient@demo.ru / 1234")

    # --- Врпач 1 ---
    if not User.query.filter_by(email="doctor@demo.ru").first():

```

```

        doctor = Doctor(
            first_name="Иван", middle_name="Иванович",
            last_name="Иванов",
            qualification="Стоматолог-терапевт", experience=12,
        )
        db.session.add(doctor)
        db.session.flush()
        u = User(email="doctor@demo.ru",
            phone="+79992223344",
            role="doctor", doctor_id=doctor.id_doctor)
        u.set_password("1234")
        db.session.add(u)
        db.session.commit()
        print(" + doctor@demo.ru / 1234")

# --- Врач 2 ---
if not User.query.filter_by(email="doctor2@demo.ru").first():
    doctor2 = Doctor(
        first_name="Анна", middle_name="Сергеевна",
        last_name="Петрова",
        qualification="Ортодонт", experience=9,
    )
    db.session.add(doctor2)
    db.session.flush()
    u2 = User(email="doctor2@demo.ru",
        phone="+79993334455",
        role="doctor", doctor_id=doctor2.id_doctor)
    u2.set_password("1234")
    db.session.add(u2)
    db.session.commit()
    print(" + doctor2@demo.ru / 1234")

# --- Менеджер ---
if not User.query.filter_by(email="manager@demo.ru").first():
    u = User(email="manager@demo.ru",
        phone="+79995556677", role="manager")
    u.set_password("1234")
    db.session.add(u)
    db.session.commit()
    print(" + manager@demo.ru / 1234")

# --- Администратор ---
if not User.query.filter_by(email="admin@demo.ru").first():
    u = User(email="admin@demo.ru",
        phone="+79998889900", role="admin")
    u.set_password("1234")
    db.session.add(u)
    db.session.commit()
    print(" + admin@demo.ru / 1234")

def migrate_existing_tables():
    """Добавляет новые колонки в уже существующие
    таблицы (безопасно)."""
    with db.engine.connect() as conn:
        migrations = [

```

```

            "ALTER TABLE appointments ADD COLUMN service_id INT NULL",
            "ALTER TABLE appointments ADD COLUMN status VARCHAR(20) NOT NULL DEFAULT 'scheduled'",
            "ALTER TABLE doctors ADD COLUMN photo VARCHAR(200) NULL",
            "ALTER TABLE users ADD COLUMN reset_token VARCHAR(64) NULL",
            "ALTER TABLE users ADD COLUMN reset_token_exp DATETIME NULL",
        ]
        for sql in migrations:
            try:
                conn.execute(db.text(sql))
                conn.commit()
                print(f" + {sql[:60]}...")
            except Exception:
                pass

if __name__ == "__main__":
    with app.app_context():
        print(" Создание таблиц...")
        db.create_all()

        print(" Миграция существующих таблиц...")
        migrate_existing_tables()

        print(" Сидер услуг...")
        seed_services()

        print(" Сидер ачивок...")
        seed_achievements()

        print(" Сидер кабинетов...")
        seed_offices()

        print(" Сидер демо-юзеров...")
        seed_demo_users()

        print("\n Готово! Теперь можно запускать app.py")

```

Листинг 3 – config.py

```

import os

class Config:
    # --- База данных ---
    DB_USER = os.getenv("DB_USER", "root")
    DB_PASSWORD = os.getenv("DB_PASSWORD",
        "123456789")
    DB_HOST = os.getenv("DB_HOST", "127.0.0.1")
    DB_PORT = os.getenv("DB_PORT", "3306")
    DB_NAME = os.getenv("DB_NAME", "ais_dentistry")

    SQLALCHEMY_DATABASE_URI = (

```

```

f"mysql+pymysql://{DB_USER}:{DB_PASS-
WORD}"
f"@{DB_HOST}:{DB_PORT}/{DB_NAME}?char-
set=utf8mb4"
)
SQLALCHEMY_TRACK_MODIFICATIONS = False

# --- Flask ---
SECRET_KEY = os.getenv("SECRET_KEY", "secret-
key")

```

Листинг 4 – app.py

```

import os
import secrets
from datetime import date, datetime, timedelta
from functools import wraps

from flask import (
    Flask, render_template, send_from_directory,
    request, redirect, url_for, session, flash, g, jsonify,
)
from werkzeug.utils import secure_filename

from config import Config
from models import (
    db, User, Patient, Doctor, Office, Appointment, Medi-
calHistory,
    Achievement, PatientAchievement, Service,
)

UPLOAD_FOLDER = os.path.join(os.path.dir-
name(__file__), "images", "doctors")
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg",
"webp"}

def create_app():
    app = Flask(__name__, template_folder="templates")
    app.config.from_object(Config)
    app.config["UPLOAD_FOLDER"] = UP-
LOAD_FOLDER
    db.init_app(app)
    register_routes(app)
    return app

```

```

# --- helpers

def current_user():
    if not hasattr(g, "_cu"):
        uid = session.get("user_id")
        g._cu = User.query.get(uid) if uid else None
    return g._cu

def login_required(view):
    @wraps(view)
    def wrapped(*a, **kw):
        if current_user() is None:
            return redirect(url_for("login"))
        return view(*a, **kw)
    return wrapped

def role_required(*roles):
    def dec(view):
        @wraps(view)
        def wrapped(*a, **kw):
            u = current_user()
            if u is None:
                return redirect(url_for("login"))
            if u.role not in roles:
                flash("Недостаточно прав.", "error")
                return redirect(url_for("cabinet"))
            return view(*a, **kw)
        return wrapped
    return dec

def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower()
in ALLOWED_EXTENSIONS

def get_patient_achievements(patient_id):
    all_ach = Achievement.query.order_by(Achieve-
ment.id_achievement).all()
    pm = {pa.achievement_id: pa
        for pa in PatientAchievement.query.filter_by(pa-
tient_id=patient_id).all()}

```

```

result = []
for ach in all_ach:
    pa = pm.get(ach.id_achievement)
    result.append({
        "title": ach.title, "desc": ach.description,
        "icon": ach.icon, "total": ach.total,
        "progress": pa.progress if pa else 0,
        "unlocked": pa.unlocked if pa else False,
    })
return result

def update_achievements(patient_id):
    visits = Appointment.query.filter_by(patient_id=pa-
tient_id).count()
    histories = MedicalHistory.query.filter_by(patient_id=pa-
tient_id).all()
    teeth = len(histories)
    cleanings = sum(1 for h in histories if h.diagnosis and
        any(w in h.diagnosis.lower() for w in
["гигиен", "чистк", "air flow"]))
    mapping = {
        "first_visit": (min(visits, 1), 1),
        "heal_1_tooth": (min(teeth, 1), 1),
        "heal_5_teeth": (min(teeth, 5), 5),
        "heal_10_teeth": (min(teeth, 10), 10),
        "clean_3": (min(cleanings, 3), 3),
        "punctual": (min(visits, 5), 5),
    }
    for code, (prog, total) in mapping.items():
        ach = Achievement.query.filter_by(code=code).first()
        if not ach:
            continue
        pa = PatientAchievement.query.filter_by(
            patient_id=patient_id, achievement_id=ach.id_achievement).first()
        if not pa:
            pa = PatientAchievement(patient_id=patient_id,
                achievement_id=ach.id_achievement)
        db.session.add(pa)
        # Если ачивка уже разблокирована — прогресс = to-
tal (не сбрасываем)
        if pa.unlocked_at:
            pa.progress = total
        else:

```

```

        pa.progress = prog
        if prog >= total:
            pa.unlocked_at = datetime.utcnow()
            db.session.commit()

def _parse_date(s):
    if not s:
        return None
    try:
        return date.fromisoformat(s)
    except ValueError:
        return None

def _appt_to_dict(a):
    return {
        "id": a.id_appointment,
        "date": a.appointment_date.strftime("%d.%m.%Y") if
a.appointment_date else "",
        "date_iso": a.appointment_date.isoformat() if a.ap-
pointment_date else "",
        "time": a.appointment_time.strftime("%H:%M") if
a.appointment_time else "",
        "doctor": a.doctor.full_name if a.doctor else "",
        "doctor_id": a.doctor_id,
        "patient": a.patient.full_name if a.patient else "",
        "patient_id": a.patient_id,
        "office": a.office.office_number if a.office else "",
        "service": a.service.name if a.service else "Приём",
        "status": a.status,
    }

def _build_stats(user, appointments):
    if user.role == "patient":
        total_v = Appointment.query.filter_by(pa-
tient_id=user.patient_id).count() if user.patient_id else 0
        hist_c = MedicalHistory.query.filter_by(pa-
tient_id=user.patient_id).count() if user.patient_id else 0
        ach_l = get_patient_achievements(user.patient_id) if
user.patient_id else []
        unlocked = sum(1 for a in ach_l if a["unlocked"])
        next_a = next((a["date"] for a in appointments if a["sta-
tus"] == "scheduled"), "")
        return {"visits": total_v, "teeth_healed": hist_c,

```

```

        "achievements": f"{unlocked} / {len(ach_1)}",
    "next_appt": next_a}
    elif user.role == "doctor":
        today = date.today()
        tc = Appointment.query.filter_by(doctor_id=user.doc-
tor_id).filter(
        Appointment.appointment_date == today).count() if
user.doctor_id else 0
        tp = db.session.query(db.func.count(db.distinct(App-
pointment.patient_id))).filter_by(
        doctor_id=user.doctor_id).scalar() if user.doctor_id
else 0
        return {"today": tc, "total_patients": tp,
        "experience": user.doctor.experience if user.doctor
else 0, "rating": "4.9"}
    elif user.role == "manager":
        today = date.today()
        return {
            "today_appts": Appointment.query.filter(Appoint-
ment.appointment_date == today).count(),
            "total_doctors": Doctor.query.count(),
            "total_patients": Patient.query.count(),
            "offices": Office.query.count(),
        }
    elif user.role == "admin":
        return {
            "users": User.query.count(),
            "doctors": Doctor.query.count(),
            "patients": Patient.query.count(),
            "appointments": Appointment.query.count(),
        }
    return {}

```

```
def register_routes(app):
```

#	—	Публичные	страницы

```

@app.route("/")
def index():
    return render_template("main.html")

@app.route("/about")
def about():

```

```

    return render_template("about.html")

@app.route("/prices")
def prices():
    services = Service.query.order_by(Service.category,
Service.name).all()
    # Группируем по категории
    categories = {}
    for s in services:
        cat = s.category or "Прочее"
        categories.setdefault(cat, []).append(s)
    return render_template("prices.html", categories=cate-
gories)

```

#	—	Авторизация

```

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        lv = request.form.get("login", "").strip()
        pw = request.form.get("password", "")
        user = User.query.filter(
            (User.email == lv) | (User.phone == lv)).first()
        if user and user.check_password(pw):
            session["user_id"] = user.id_user
            return redirect(url_for("cabinet"))
        return render_template("login.html", er-
ror="Неверный логин или пароль")
    return render_template("login.html")

```

```
@app.route("/register", methods=["GET", "POST"])
```

```

def register():
    if request.method == "POST":
        f = request.form
        errors = {}
        if not f.get("first_name", "").strip():
            errors["first_name"] = "Введите имя"
        if not f.get("last_name", "").strip():
            errors["last_name"] = "Введите фамилию"
        email = f.get("email", "").strip()
        if not email or "@" not in email:
            errors["email"] = "Введите корректный email"
        elif User.query.filter_by(email=email).first():
            errors["email"] = "Email уже зарегистрирован"

```

```

phone = f.get("phone", "").strip()
if not phone:
    errors["phone"] = "Введите телефон"
pw = f.get("password", "")
pw2 = f.get("password2", "")
if len(pw) < 6:
    errors["password"] = "Минимум 6 символов"
elif pw != pw2:
    errors["password2"] = "Пароли не совпадают"
if errors:
    return render_template("register.html", errors=errors, form=f)

patient = Patient(
    first_name=f.get("first_name"),
    middle_name=f.get("middle_name"),
    last_name=f.get("last_name"),
    phone=phone,
    birth_date=_parse_date(f.get("birth_date")),
)
db.session.add(patient)
db.session.flush()
user = User(email=email, phone=phone, role="patient",
            patient_id=patient.id_patient)
user.set_password(pw)
db.session.add(user)
db.session.commit()
session["user_id"] = user.id_user
flash("Добро пожаловать! Аккаунт создан.", "success")
return redirect(url_for("cabinet"))
return render_template("register.html", errors={}, form={})

@app.route("/logout")
def logout():
    session.pop("user_id", None)
    return redirect(url_for("index"))

# — Восстановление пароля

@app.route("/forgot-password", methods=["GET", "POST"])

```

```

def forgot_password():
    if request.method == "POST":
        email = request.form.get("email", "").strip()
        user = User.query.filter_by(email=email).first()
        if user:
            token = secrets.token_hex(32)
            user.reset_token = token
            user.reset_token_exp = datetime.utcnow() + timedelta(hours=1)
            db.session.commit()
            reset_url = url_for("reset_password", token=token, _external=True)
            flash(f"Ссылка для сброса пароля (скопируйте): {reset_url}", "info")
        else:
            flash("Если такой email зарегистрирован, ссылка отправлена.", "info")
            return redirect(url_for("forgot_password"))
            return render_template("forgot_password.html")

@app.route("/reset-password/<token>", methods=["GET", "POST"])
def reset_password(token):
    user = User.query.filter_by(reset_token=token).first()
    if not user or not user.reset_token_exp or user.reset_token_exp < datetime.utcnow():
        flash("Ссылка недействительна или устарела.", "error")
        return redirect(url_for("login"))
    if request.method == "POST":
        pw = request.form.get("password", "")
        pw2 = request.form.get("password2", "")
        if len(pw) < 6:
            return render_template("reset_password.html", token=token,
                                error="Минимум 6 символов")
        if pw != pw2:
            return render_template("reset_password.html", token=token,
                                error="Пароли не совпадают")
        user.set_password(pw)
        user.reset_token = None
        user.reset_token_exp = None
        db.session.commit()
        flash("Пароль успешно изменён. Войдите с новым паролем.", "success")

```

```

        return redirect(url_for("login"))
    return render_template("reset_password.html", token=token, error=None)

```

```

#          ————— Личный кабинет

```

```

def _cabinet_common_ctx(user):
    """Общий контекст для всех страниц кабинета."""
    notifications = []
    if user.role == "patient" and user.patient_id:
        upcoming = Appointment.query.filter_by(
            patient_id=user.patient_id, status="scheduled"
        ).filter(
            Appointment.appointment_date >= date.today(),
            Appointment.appointment_date <= date.today() +
timedelta(days=2)
        ).order_by(Appointment.appointment_date, Appointment.appointment_time).first()
        if upcoming:
            notifications.append({
                "type": "info",
                "text": f"Напоминание: у вас запись {upcoming.appointment_date.strftime('%d.%m.%Y')} в {upcoming.appointment_time.strftime('%H:%M')} if upcoming.appointment_time else '—'"
                f" к врачу {upcoming.doctor.full_name if upcoming.doctor else '—'}."
            })
        elif user.role == "doctor" and user.doctor_id:
            today_count = Appointment.query.filter_by(
                doctor_id=user.doctor_id, status="scheduled"
            ).filter(Appointment.appointment_date == date.today()).count()
            if today_count:
                notifications.append({
                    "type": "info",
                    "text": f"Сегодня у вас {today_count} запланированных приёмов."
                })

    user_ctx = {
        "first_name": user.first_name,
        "middle_name": user.middle_name,

```

```

        "last_name": user.last_name,
        "phone": user.phone or (user.patient.phone if user.patient else ""),
        "email": user.email,
        "role": user.role,
        "role_label": user.role_label,
        "experience": user.doctor.experience if user.doctor else None,
        "qualification": user.doctor.qualification if user.doctor else None,
        "photo": user.doctor.photo if user.doctor else None,
    }
    return {"user": user_ctx, "notifications": notifications}

@app.route("/cabinet")
@login_required
def cabinet():
    user = current_user()
    ctx = _cabinet_common_ctx(user)

    appts_q = Appointment.query
    if user.role == "patient" and user.patient_id:
        appts_q = appts_q.filter_by(patient_id=user.patient_id)
    elif user.role == "doctor" and user.doctor_id:
        appts_q = appts_q.filter_by(doctor_id=user.doctor_id)
    appointments = [_appt_to_dict(a) for a in appts_q.order_by(Appointment.appointment_date, Appointment.appointment_time).limit(30).all()]

    if user.role == "patient" and user.patient_id:
        update_achievements(user.patient_id)

    stats = _build_stats(user, appointments)
    ctx.update({
        "stats": stats,
        "active_page": "dashboard",
    })
    return render_template("cabinet_dashboard.html", **ctx)

@app.route("/cabinet/profile")
@login_required

```

```

def cabinet_profile():
    user = current_user()
    ctx = _cabinet_common_ctx(user)
    ctx["active_page"] = "profile"
    return render_template("cabinet_profile.html", **ctx)

@app.route("/cabinet/appointments")
@login_required
def cabinet_appointments():
    user = current_user()
    ctx = _cabinet_common_ctx(user)

    appts_q = Appointment.query
    if user.role == "patient" and user.patient_id:
        appts_q = appts_q.filter_by(patient_id=user.pa-
tient_id)
    elif user.role == "doctor" and user.doctor_id:
        appts_q = appts_q.filter_by(doctor_id=user.doc-
tor_id)
    appointments = [_appt_to_dict(a) for a in
        appts_q.order_by(Appointment.appoint-
ment_date,
                        Appointment.appoint-
ment_time).limit(50).all()]

    doctors = Doctor.query.order_by(Doc-
tor.last_name).all()
    offices = Office.query.order_by(Office.office_num-
ber).all()
    services = Service.query.order_by(Service.name).all()
    patients = Patient.query.order_by(Pa-
tient.last_name).all() \
        if user.role in ("manager", "admin", "doctor") else []

    ctx.update({
        "appointments": appointments,
        "doctors": doctors,
        "offices": offices,
        "services": services,
        "patients": patients,
        "today_iso": date.today().isoformat(),
        "active_page": "appointments",
    })
    return render_template("cabinet_appointments.html",
**ctx)

```

```

@app.route("/cabinet/history")
@login_required
def cabinet_history():
    user = current_user()
    ctx = _cabinet_common_ctx(user)

    histories = []
    hq = MedicalHistory.query
    if user.role == "patient" and user.patient_id:
        hq = hq.filter_by(patient_id=user.patient_id)
    elif user.role == "doctor" and user.doctor_id:
        hq = hq.filter_by(doctor_id=user.doctor_id)
    for h in hq.order_by(MedicalHis-
tory.visit_date.desc()).limit(50).all():
        histories.append({
            "id": h.id_history,
            "date": h.visit_date.strftime("%d.%m.%Y") if
h.visit_date else "—",
            "diagnosis": h.diagnosis or "—",
            "treatment": h.treatment or "—",
            "doctor": h.doctor.full_name if h.doctor else "—",
            "patient": h.patient.full_name if h.patient else "—",
        })

    doctors = Doctor.query.order_by(Doc-
tor.last_name).all() \
        if user.role in ("manager", "admin") else []
    patients = Patient.query.order_by(Pa-
tient.last_name).all() \
        if user.role in ("manager", "admin", "doctor") else []

    ctx.update({
        "histories": histories,
        "doctors": doctors,
        "patients": patients,
        "active_page": "history",
    })
    return render_template("cabinet_history.html", **ctx)

@app.route("/cabinet/achievements")
@login_required
def cabinet_achievements():
    user = current_user()
    if user.role != "patient":

```

```

        flash("Достижения доступны только для пациен-
тов.", "error")
        return redirect(url_for("cabinet"))
    ctx = _cabinet_common_ctx(user)
    if user.patient_id:
        update_achievements(user.patient_id)
        achievements = get_patient_achievements(user.pa-
tient_id)
    else:
        achievements = []
    ctx.update({
        "achievements": achievements,
        "active_page": "achievements",
    })
    return render_template("cabinet_achievements.html",
**ctx)

@app.route("/cabinet/manage")
@login_required
def cabinet_manage():
    user = current_user()
    if user.role not in ("manager", "admin"):
        flash("Недостаточно прав.", "error")
        return redirect(url_for("cabinet"))
    ctx = _cabinet_common_ctx(user)
    ctx["active_page"] = "manage"
    return render_template("cabinet_manage.html", **ctx)

@app.route("/cabinet/prices")
@login_required
def cabinet_prices():
    user = current_user()
    ctx = _cabinet_common_ctx(user)
    price_categories = {}
    for s in Service.query.order_by(Service.category, Ser-
vice.name).all():
        cat = s.category or "Прочее"
        price_categories.setdefault(cat, []).append(s)
    ctx.update({
        "price_categories": price_categories,
        "active_page": "prices",
    })
    return render_template("cabinet_prices.html", **ctx)

@app.route("/cabinet/settings")
@login_required

```

```

def cabinet_settings():
    user = current_user()
    ctx = _cabinet_common_ctx(user)
    ctx["active_page"] = "settings"
    return render_template("cabinet_settings.html", **ctx)

@app.route("/cabinet/legacy")
@login_required
def cabinet_legacy():
    user = current_user()

    # Уведомление о ближайшем приёме
    notifications = []
    if user.role == "patient" and user.patient_id:
        upcoming = Appointment.query.filter_by(
            patient_id=user.patient_id, status="scheduled"
        ).filter(
            Appointment.appointment_date >= date.today(),
            Appointment.appointment_date <= date.today() +
timedelta(days=2)
        ).order_by(Appointment.appointment_date, Ap-
pointment.appointment_time).first()
        if upcoming:
            notifications.append({
                "type": "info",
                "text": f"Напоминание: у вас запись {upcom-
ing.appointment_date.strftime('%d.%m.%Y')}}"
                    f" в {upcoming.appoint-
ment_time.strftime('%H:%M')} if upcoming.appoint-
ment_time else '—'"
                    f" к врачу {upcoming.doctor.full_name if
upcoming.doctor else '—'}."
            })
        elif user.role == "doctor" and user.doctor_id:
            today_count = Appointment.query.filter_by(
                doctor_id=user.doctor_id, status="scheduled"
            ).filter(Appointment.appointment_date == date.to-
day()).count()
            if today_count:
                notifications.append({
                    "type": "info",
                    "text": f"Сегодня у вас {today_count} запла-
нированных приёмов."
                })

    # Записи

```

```

appts_q = Appointment.query
if user.role == "patient" and user.patient_id:
    appts_q = appts_q.filter_by(patient_id=user.pa-
tient_id)
elif user.role == "doctor" and user.doctor_id:
    appts_q = appts_q.filter_by(doctor_id=user.doc-
tor_id)
appointments = [_ appt_to_dict(a) for a in
    appts_q.order_by(Appointment.appoint-
ment_date,
                    Appointment.appoint-
ment_time).limit(30).all()]

# История
histories = []
hq = MedicalHistory.query
if user.role == "patient" and user.patient_id:
    hq = hq.filter_by(patient_id=user.patient_id)
elif user.role == "doctor" and user.doctor_id:
    hq = hq.filter_by(doctor_id=user.doctor_id)
for h in hq.order_by(MedicalHis-
tory.visit_date.desc()).limit(30).all():
    histories.append({
        "id": h.id_history,
        "date": h.visit_date.strftime("%d.%m.%Y") if
h.visit_date else "—",
        "diagnosis": h.diagnosis or "—",
        "treatment": h.treatment or "—",
        "doctor": h.doctor.full_name if h.doctor else "—",
        "patient": h.patient.full_name if h.patient else "—
",
    })

# Ачивки
achievements = []
if user.role == "patient" and user.patient_id:
    update_achievements(user.patient_id)
    achievements = get_patient_achievements(user.pa-
tient_id)

stats = _build_stats(user, appointments)
doctors = Doctor.query.order_by(Doc-
tor.last_name).all()
offices = Office.query.order_by(Office.office_num-
ber).all()
services = Service.query.order_by(Service.name).all()

```

```

patients = Patient.query.order_by(Pa-
tient.last_name).all() \
    if user.role in ("manager", "admin", "doctor") else []

# Прайс-лист сгруппированный по категориям
price_categories = {}
for s in Service.query.order_by(Service.category, Ser-
vice.name).all():
    cat = s.category or "Прочее"
    price_categories.setdefault(cat, []).append(s)

ctx = {
    "user": {
        "first_name": user.first_name,
        "middle_name": user.middle_name,
        "last_name": user.last_name,
        "phone": user.phone or (user.patient.phone if
user.patient else ""),
        "email": user.email,
        "role": user.role,
        "role_label": user.role_label,
        "experience": user.doctor.experience if user.doctor
else None,
        "qualification": user.doctor.qualification if
user.doctor else None,
        "photo": user.doctor.photo if user.doctor else
None,
    },
    "achievements": achievements,
    "appointments": appointments,
    "histories": histories,
    "stats": stats,
    "doctors": doctors,
    "offices": offices,
    "services": services,
    "patients": patients,
    "today_iso": date.today().isoformat(),
    "notifications": notifications,
    "price_categories": price_categories,
}

return render_template("cabinet.html", **ctx)

```

— Завершение приёма (врач)

```

@app.route("/cabinet/appointment/<int:appt_id>/complete", methods=["POST"])
@login_required
def cabinet_appointment_complete(appt_id):
    user = current_user()
    appt = Appointment.query.get_or_404(appt_id)
    if user.role == "doctor" and appt.doctor_id != user.doctor_id:
        flash("Нет доступа.", "error")
        return redirect(url_for("cabinet_appointments"))
    if user.role not in ("doctor", "manager", "admin"):
        flash("Нет доступа.", "error")
        return redirect(url_for("cabinet_appointments"))
    appt.status = "completed"
    db.session.commit()
    flash("Приём завершён.", "success")
    return redirect(url_for("cabinet_appointments"))

```

#	—	Новая	запись

```

@app.route("/cabinet/appointment/new", methods=["POST"])
@login_required
def cabinet_appointment_new():
    user = current_user()
    f = request.form
    doctor_id = f.get("doctor_id") or None
    office_id = f.get("office_id") or None
    service_id = f.get("service_id") or None
    appt_date = _parse_date(f.get("appointment_date"))
    appt_time = None
    ts = f.get("appointment_time", "").strip()
    if ts:
        try:
            from datetime import time as dtime
            p = ts.split(":")
            appt_time = dtime(int(p[0]), int(p[1]))
        except Exception:
            pass
    patient_id = user.patient_id if user.role == "patient" else (f.get("patient_id") or None)
    if not patient_id or not appt_date:
        flash("Заполните обязательные поля: пациент и дата.", "error")

```

```

        return redirect(url_for("cabinet_appointments"))
    db.session.add(Appointment(
        patient_id=patient_id, doctor_id=doctor_id, office_id=office_id,
        service_id=service_id, appointment_date=appt_date,
        appointment_time=appt_time, status="scheduled",
    ))
    db.session.commit()
    flash("Запись успешно создана.", "success")
    return redirect(url_for("cabinet_appointments"))

```

#	—	Отмена	записи

```

@app.route("/cabinet/appointment/<int:appt_id>/cancel", methods=["POST"])
@login_required
def cabinet_appointment_cancel(appt_id):
    user = current_user()
    appt = Appointment.query.get_or_404(appt_id)
    if user.role == "patient" and appt.patient_id != user.patient_id:
        flash("Нет доступа.", "error")
        return redirect(url_for("cabinet_appointments"))
    appt.status = "cancelled"
    db.session.commit()
    flash("Запись отменена.", "success")
    return redirect(url_for("cabinet_appointments"))

```

#	—	История	лечения

```

@app.route("/cabinet/history/add", methods=["POST"])
@login_required
@role_required("doctor", "manager", "admin")
def cabinet_history_add():
    f = request.form
    user = current_user()
    patient_id = f.get("patient_id") or None
    doctor_id = f.get("doctor_id") or (user.doctor_id if user.role == "doctor" else None)
    visit_date = _parse_date(f.get("visit_date"))
    if not patient_id or not visit_date:

```

```

        flash("Заполните обязательные поля.", "error")
        return redirect(url_for("cabinet_history"))
    db.session.add(MedicalHistory(
        patient_id=patient_id, doctor_id=doctor_id,
        diagnosis=f.get("diagnosis", "").strip(),
        treatment=f.get("treatment", "").strip(),
        visit_date=visit_date,
    ))
    db.session.commit()
    update_achievements(int(patient_id))
    flash("Запись в историю добавлена.", "success")
    return redirect(url_for("cabinet_history"))

# ————— Профиль

```

```

@app.route("/cabinet/profile/edit", methods=["POST"])
@login_required
def cabinet_profile_edit():
    user = current_user()
    f = request.form
    if user.patient:
        user.patient.first_name = f.get("first_name") or
user.patient.first_name
        user.patient.middle_name = f.get("middle_name") or
user.patient.middle_name
        user.patient.last_name = f.get("last_name") or
user.patient.last_name
        user.patient.phone = f.get("phone") or user.pa-
tient.phone
        bd = _parse_date(f.get("birth_date"))
        if bd:
            user.patient.birth_date = bd
    elif user.doctor:
        user.doctor.first_name = f.get("first_name") or
user.doctor.first_name
        user.doctor.middle_name = f.get("middle_name") or
user.doctor.middle_name
        user.doctor.last_name = f.get("last_name") or
user.doctor.last_name
        new_email = f.get("email", "").strip()
        if new_email and new_email != user.email:
            if User.query.filter_by(email=new_email).first():
                flash("Этот email уже занят.", "error")
            return redirect(url_for("cabinet_profile"))

```

```

        user.email = new_email
    if f.get("phone"):
        user.phone = f.get("phone")
    db.session.commit()
    flash("Профиль обновлён.", "success")
    return redirect(url_for("cabinet_profile"))

# ————— Загрузка фото врача

```

```

@app.route("/cabinet/doctor/photo", methods=["POST"])
@login_required
def cabinet_doctor_photo():
    user = current_user()
    if not user.doctor:
        flash("Только врачи могут загружать фото.",
"error")
        return redirect(url_for("cabinet_profile"))
    file = request.files.get("photo")
    if not file or file.filename == "":
        flash("Файл не выбран.", "error")
        return redirect(url_for("cabinet_profile"))
    if not allowed_file(file.filename):
        flash("Допустимые форматы: PNG, JPG, JPEG,
WEBP.", "error")
        return redirect(url_for("cabinet_profile"))
    ext = file.filename.rsplit(".", 1)[1].lower()
    filename = secure_filename(f'doctor_{user.doc-
tor_id}.{ext}')
    file.save(os.path.join(app.config["UP-
LOAD_FOLDER"], filename))
    user.doctor.photo = filename
    db.session.commit()
    flash("Фото обновлено.", "success")
    return redirect(url_for("cabinet_profile"))

# ————— Смена пароля

```

```

@app.route("/cabinet/password/change", methods=["POST"])
@login_required
def cabinet_password_change():

```

```

user = current_user()
f = request.form
if not user.check_password(f.get("old_password", "")):
    flash("Неверный текущий пароль.", "error")
    return redirect(url_for("cabinet_settings"))
pw = f.get("new_password", "")
if len(pw) < 6:
    flash("Новый пароль — минимум 6 символов.",
"error")
    return redirect(url_for("cabinet_settings"))
if pw != f.get("new_password2", ""):
    flash("Пароли не совпадают.", "error")
    return redirect(url_for("cabinet_settings"))
user.set_password(pw)
db.session.commit()
flash("Пароль изменён.", "success")
return redirect(url_for("cabinet_settings"))

```

#	—	API:	слоты

```

@app.route("/api/slots")
@login_required
def api_slots():
    doctor_id = request.args.get("doctor_id")
    appt_date = request.args.get("date")
    if not doctor_id or not appt_date:
        return jsonify([])
    busy = {
        a.appointment_time.strftime("%H:%M")
        for a in Appointment.query.filter_by(
            doctor_id=doctor_id,
            appointment_date=_parse_date(appt_date)
        ).filter(Appointment.status != "cancelled").all()
        if a.appointment_time
    }
    slots = [f'{h:02d}:{m:02d}' for h in range(8, 20) for m
in (0, 30)]
    return jsonify([s for s in slots if s not in busy])

```

#	—	API:	поиск	пациентов

```
@app.route("/cabinet/patients")
```

```

@login_required
@role_required("manager", "admin", "doctor")
def cabinet_patients():
    q = request.args.get("q", "").strip()
    query = Patient.query
    if q:
        like = f"%{q}%"
        query = query.filter(
            db.or_(
                Patient.last_name.ilike(like),
                Patient.first_name.ilike(like),
                Patient.phone.ilike(like),
            )
        )
    patients = query.order_by(Pa-
tient.last_name).limit(100).all()
    result = []
    for p in patients:
        last_h = MedicalHistory.query.filter_by(pa-
tient_id=p.id_patient)\
            .order_by(MedicalHis-
tory.visit_date.desc()).first()
        result.append({
            "id": p.id_patient,
            "name": p.full_name,
            "phone": p.phone or "—",
            "birth_date": p.birth_date.strftime("%d.%m.%Y")
if p.birth_date else "—",
            "visits": Appointment.query.filter_by(pa-
tient_id=p.id_patient).count(),
            "last_visit":
last_h.visit_date.strftime("%d.%m.%Y")
if last_h and last_h.visit_date else "—",
        })
    return jsonify(result)

```

#	—	API:	поиск	записей

```

@app.route("/api/appointments")
@login_required
@role_required("manager", "admin")
def api_appointments():
    q = request.args.get("q", "").strip()
    status = request.args.get("status", "")

```

```

query = Appointment.query
if status:
    query = query.filter_by(status=status)
if q:
    like = f"%{q}%"
    query = query.join(Patient, isouter=True).join(Doctor, isouter=True).filter(
        db.or_(
            Patient.last_name.ilike(like),
            Patient.first_name.ilike(like),
            Doctor.last_name.ilike(like),
        )
    )
appts = query.order_by(Appointment.appointment_date.desc()).limit(50).all()
return jsonify([_appt_to_dict(a) for a in appts])

```

— Управление: добавить врача

```

@app.route("/cabinet/doctor/add", methods=["POST"])
@login_required
@role_required("manager", "admin")
def cabinet_doctor_add():
    f = request.form
    email = f.get("email", "").strip()
    if not email:
        flash("Email обязателен.", "error")
        return redirect(url_for("cabinet_manage"))
    if User.query.filter_by(email=email).first():
        flash("Пользователь с таким email уже существует.", "error")
        return redirect(url_for("cabinet_manage"))

    # Роль: admin может выбрать любую, менеджер — только doctor
    cur = current_user()
    role_override = f.get("role_override", "doctor")
    if cur.role != "admin":
        role_override = "doctor"

    doctor = None
    if role_override == "doctor":
        doctor = Doctor(

```

```

            first_name=f.get("first_name"),
            middle_name=f.get("middle_name"),
            last_name=f.get("last_name"),
            qualification=f.get("qualification"),
            experience=int(f.get("experience") or 0),
        )
        db.session.add(doctor)
        db.session.flush()

    u = User(
        email=email,
        phone=f.get("phone"),
        role=role_override,
        doctor_id=doctor.id_doctor if doctor else None,
    )
    u.set_password(f.get("password") or "changeme")
    db.session.add(u)
    db.session.commit()

```

```

    name = doctor.full_name if doctor else email
    flash(f"Сотрудник {name} добавлен с ролью «{role_override}».", "success")
    return redirect(url_for("cabinet_manage"))

```

— Управление: удалить пользователя (только admin)

```

@app.route("/cabinet/user/<int:uid>/delete", methods=["POST"])
@login_required
@role_required("admin")
def cabinet_user_delete(uid):
    user = current_user()
    if uid == user.id_user:
        flash("Нельзя удалить себя.", "error")
        return redirect(url_for("cabinet_manage"))
    u = User.query.get_or_404(uid)
    db.session.delete(u)
    db.session.commit()
    flash("Пользователь удалён.", "success")
    return redirect(url_for("cabinet_manage"))

```

— Управление: список всех пользователей (admin)

```

@app.route("/api/users")

```

```

@login_required
@role_required("admin")
def api_users():
    users = User.query.order_by(User.role,
User.email).all()
    return jsonify([
        "id": u.id_user,
        "email": u.email,
        "phone": u.phone or "—",
        "role": u.role,
        "role_label": u.role_label,
        "name": f'{u.last_name} {u.first_name}'.strip() or
u.email,
        "created": u.created_at.strftime("%d.%m.%Y") if
u.created_at else "—",
    ] for u in users])

```

```

# — Карточка пациента

```

```

@app.route("/patient/<int:patient_id>")
@login_required
@role_required("doctor", "manager", "admin")
def patient_card(patient_id):
    patient = Patient.query.get_or_404(patient_id)
    histories = []
    for h in MedicalHistory.query.filter_by(patient_id=pa-
tient_id)\
        .order_by(MedicalHistory.visit_date.desc()).all():
        histories.append({
            "date": h.visit_date.strftime("%d.%m.%Y") if
h.visit_date else "—",
            "diagnosis": h.diagnosis or "—",
            "treatment": h.treatment or "—",
            "doctor": h.doctor.full_name if h.doctor else "—",
        })
    appointments = [_appt_to_dict(a) for a in
Appointment.query.filter_by(patient_id=pa-
tient_id)
        .order_by(Appointment.appoint-
ment_date.desc()).all()]
    return render_template("patient_card.html",
        patient=patient,
        histories=histories,
        appointments=appointments)

```

```

@app.errorhandler(404)
def page_not_found(e):
    return render_template("404.html"), 404

@app.errorhandler(403)
def forbidden(e):
    return render_template("404.html", code=403,
        message="Доступ запрещён"), 403

# — Статика

```

```

@app.route("/css/<path:filename>")
def serve_css(filename): return send_from_direct-
tory("css", filename)

```

```

@app.route("/js/<path:filename>")
def serve_js(filename): return send_from_directory("js",
filename)

```

```

@app.route("/images/<path:filename>")
def serve_images(filename): return send_from_direct-
tory("images", filename)

```

```

app = create_app()

if __name__ == "__main__":
    app.run(debug=True)

```

Листинг 5 – carousel.js

```

(function () {
    'use strict';

    function initCarousel(root) {
        const viewport = root.querySelector('[data-carousel-
viewport]');
        const track = root.querySelector('[data-carousel-
track]');
        const prevBtn = root.querySelector('[data-carousel-
prev]');
        const nextBtn = root.querySelector('[data-carousel-
next]');
    }

```

```

const dotsBox = root.querySelector('[data-carousel-
dots]');
const slides = Array.from(track.children);

if (!slides.length) return;

let index = 0;

// Сколько слайдов помещается за раз — вычисляем
по реальной ширине.
function slidesPerView() {
  const slideW = slides[0].getBoundingClien-
tRect().width;
  const gap = parseFloat(getComput-
edStyle(track).gap) || 0;
  const viewportW = viewport.getBoundingClien-
tRect().width;
  return Math.max(1, Math.round((viewportW + gap)
/ (slideW + gap)));
}

function maxIndex() {
  return Math.max(0, slides.length - slidesPerView());
}

function update() {
  const slideW = slides[0].getBoundingClien-
tRect().width;
  const gap = parseFloat(getComput-
edStyle(track).gap) || 0;
  const offset = index * (slideW + gap);
  track.style.transform = `translateX(-${offset}px)`;

  if (prevBtn) prevBtn.disabled = index <= 0;
  if (nextBtn) nextBtn.disabled = index >= max-
Index();

  // Подсветка точек
  if (dotsBox) {
    Array.from(dotsBox.children).forEach((dot, i) =>
{
      dot.classList.toggle('active', i === index);
    });
  }
}

```

```

function go(i) {
  index = Math.max(0, Math.min(i, maxIndex()));
  update();
}

// Точки
function buildDots() {
  if (!dotsBox) return;
  dotsBox.innerHTML = "";
  const total = maxIndex() + 1;
  for (let i = 0; i < total; i++) {
    const dot = document.createElement('button');
    dot.className = 'carousel-dot';
    dot.type = 'button';
    dot.setAttribute('aria-label', `Слайд ${i + 1}`);
    dot.addEventListener('click', () => go(i));
    dotsBox.appendChild(dot);
  }
}

// Кнопки
if (prevBtn) prevBtn.addEventListener('click', () =>
go(index - 1));
if (nextBtn) nextBtn.addEventListener('click', () =>
go(index + 1));

// Клавиатура (когда фокус внутри карусели)
root.addEventListener('keydown', (e) => {
  if (e.key === 'ArrowLeft') { e.preventDefault();
go(index - 1); }
  if (e.key === 'ArrowRight') { e.preventDefault();
go(index + 1); }
});
root.tabIndex = 0;

// Свайп / drag
let startX = 0;
let dragging = false;

viewport.addEventListener('pointerdown', (e) => {
  dragging = true;
  startX = e.clientX;
  viewport.setPointerCapture(e.pointerId);
});
viewport.addEventListener('pointerup', (e) => {
  if (!dragging) return;

```

```

    dragging = false;
    const dx = e.clientX - startX;
    if (Math.abs(dx) > 40) {
      go(index + (dx < 0 ? 1 : -1));
    }
  });
  viewport.addEventListener('pointercancel', () => {
    dragging = false; });

```

// Ресайз — пересчитать точки и положение

```

let resizeTimer;
window.addEventListener('resize', () => {
  clearTimeout(resizeTimer);
  resizeTimer = setTimeout(() => {
    buildDots();
    go(index); // проверит границы
  }, 150);
});

buildDots();
update();
}

document.addEventListener('DOMContentLoaded', () =>
{
  document.querySelectorAll('[data-carousel]').fo-
rEach(initCarousel);
});
})0;

```